WEEK 02 · 2주차

빅데이터 개론과 생성형 AI 활용

AI기반 빅데이터 프로젝트 · 2026-2

승인된 수강생의 학습을 위한 자료입니다. 퀴즈 정답과 해설은 교수자의 별도 공개 전까지 포함하지 않습니다.

2장. 빅데이터 개론과 생성형 AI 활용

1. 학습목표

1. (데이터분석활용) 빅데이터의 5V(Volume, Velocity, Variety, Veracity, Value)를 정의하고 실제 데이터셋의 속성에 대응시켜 설명할 수 있다.
2. (데이터분석활용) 스케일업(scale-up)과 스케일아웃(scale-out)의 비용·성능 구조를 비교하고, 분산 처리가 필요해지는 임계점을 정량적 근거와 함께 제시할 수 있다.
3. (신기술융합) Hadoop, Spark, Kafka, Elastic Stack이 데이터 파이프라인의 어느 계층에 위치하는지 설명할 수 있다.
4. (신기술융합) 생성형 AI에 구조화 프롬프트로 파이프라인 설계 초안을 요청하고, 산출물의 결함을 검증 절차에 따라 식별·문서화할 수 있다.
5. (시스템운영) 운영체제의 프로세스·메모리·파일시스템 관리가 Hadoop·Spark 작업의 실행과 저장 성능에 어떻게 연결되는지 설명할 수 있다.

---

2. 내용

2.1 빅데이터 5V와 규모의 임계점

빅데이터는 절대적 용량으로 정의되지 않는다. 동일한 100 GB 데이터라도 하루에 한 번 조회하는 정적 아카이브와 초당 수만 건이 유입되는 로그 스트림은 요구되는 기술 스택이 전혀 다르다. 따라서 데이터의 성격을 다섯 축으로 나누어 진단하는 5V 관점이 사용된다.

Volume은 저장·처리해야 하는 데이터의 총량이다. Velocity는 데이터가 생성되고 소비되는 속도이며, 배치(batch)와 스트리밍(streaming)의 선택 기준이 된다. Variety는 정형·반정형·비정형이 혼재하는 구조적 다양성이다. Veracity는 결측·중복·이상값을 포함한 데이터의 신뢰도이며, 분석 결과의 타당성을 좌우한다. Value는 앞의 네 축에 투입한 비용을 정당화하는 최종 산출 가치이다.

> 〔개념〕 5V는 측정 대상이 아니라 진단 축이다. 각 축마다 관측 가능한 측정지표를 정하고 임계치를 넘는 축이 무엇인지 판정해야 기술 선택의 근거가 확보된다.

표 2-1. 5V의 정의·측정지표·대응 기술

축	정의	측정지표	임계 징후	대응 기술
Volume	저장·처리 총량	일일 증가량(GB/일), 누적 용량(TB)	단일 노드 디스크·메모리 초과	HDFS, Amazon S3, 컬럼 포맷(Parquet)
Velocity	유입·처리 속도	초당 이벤트 수(EPS), 종단 지연(초)	수집 지연 누적, 큐 적체	Kafka, Spark Structured Streaming
Variety	구조적 다양성	소스 수, 스키마 버전 수, 비정형 비율	스키마 변경마다 파이프라인 중단	스키마 레지스트리, Elasticsearch
Veracity	신뢰도	결측률, 중복률, 이상값 비율	지표가 재계산할 때마다 달라짐	데이터 품질 검증, Great Expectations류 규칙

축	정의	측정지표	임계 징후	대응 기술
Value	산출 가치	의사결정 반영 건수, 비용 대비 효과	파이프라인은 도는데 사용처가 없음	대시보드, ML 모델, 리포트

그림 2-1. 빅데이터 5V 방사형 도식 — 중심의 데이터 자산을 다섯 축이 둘러싸며, 각 축이 임계치를 넘을 때 어떤 기술 계층이 호출되는지 보여준다. (인터랙티브 도식은 03_도식_HTML/CH02_빅데이터개론과생성형AI활용.html 참조)

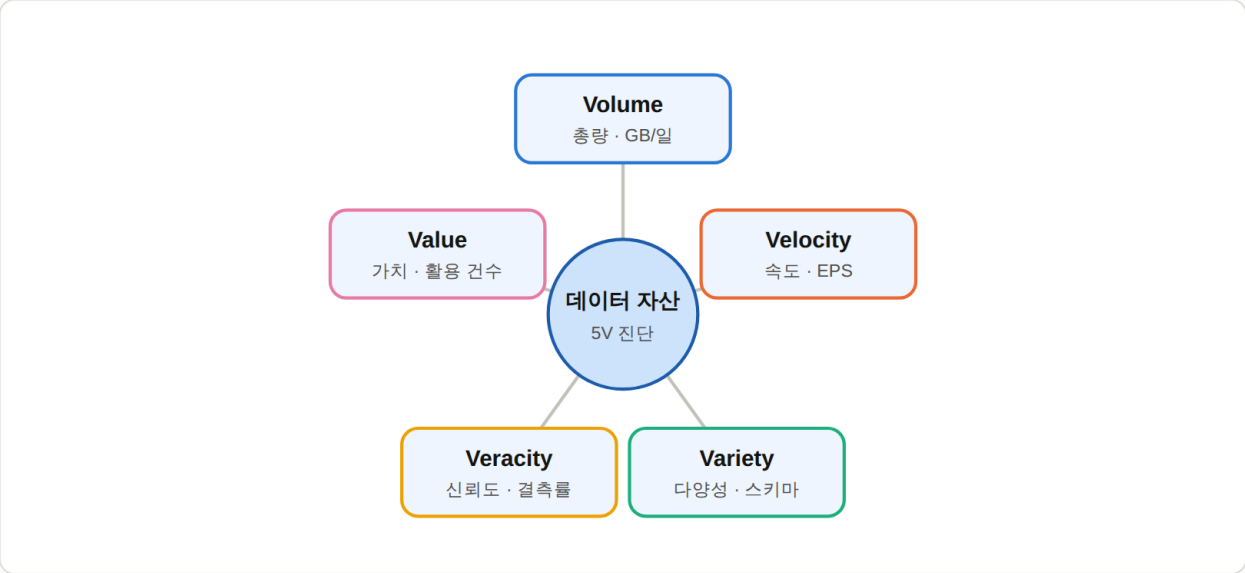


그림 2-1. 빅데이터 5V 방사형 도식

데이터의 규모를 진단하는 첫 절차는 측정이다. 추정치가 아니라 실제 용량과 행 수를 확인해야 한다.

```
du -sh /data/logs
find /data/logs -name '*.csv' | wc -l
head -3 /data/logs/access-2026-09-01.csv

import pandas as pd

df = pd.read_csv("/data/logs/access-2026-09-01.csv")
print("행/열:", df.shape)
print("메모리:", round(df.memory_usage(deep=True).sum() / 1024**2, 1), "MB")
```

코드 2-1. 데이터 규모 실측. 파일 시스템 상의 용량과 메모리 적재 시 용량이 다르므로 두 값을 함께 확인한다.

2.2 스케일업 대 스케일아웃, 분산 처리가 필요한 지점

처리 능력을 늘리는 방법은 두 가지이다. 스케일업은 단일 노드의 CPU·메모리·디스크를 상위 사양으로 교체하는 방식이고, 스케일아웃은 동일 사양의 노드를 여러 대 추가하는 방식이다.

스케일업은 코드 변경이 필요 없다는 장점이 있다. 기존 단일 프로세스 프로그램이 그대로 동작하며 분산 처리에 따르는 네트워크 셔플, 부분 실패, 데이터 지역성 문제를 고려하지 않아도 된다. 그러나 단일 장비의 최대 사양에는 물리적 상한이 있고, 사양이 올라갈수록 단가가 선형이 아니라 급격히 증가한다. 또한 단일 장애점(single point of failure)이 그대로 남는다.

스케일아웃은 상용 하드웨어를 수평으로 늘려 상한을 사실상 제거한다. 대신 프로그램이 분산 실행 모델을 전제로 작성되어야 하고, 노드 간 데이터 이동 비용과 부분 실패 처리가 새로운 설계 변수로 추가된다.

표 2-2. 스케일업과 스케일아웃 비교

구분	스케일업(Scale-up)	스케일아웃(Scale-out)
증설 방식	단일 노드 사양 상향	동일 사양 노드 추가
성능 상한	장비 최대 사양에 종속	노드 수에 비례(셔플 비용 제외)
비용 곡선	고사양 구간에서 급증	대체로 선형
코드 영향	없음	분산 실행 모델로 재작성 필요
장애 내성	단일 장애점 존재	복제·재시도로 부분 실패 흡수
고정 오버헤드	없음	작업 스케줄링·셔플에 수십 초 단위 발생
적합 구간	수 GB~수십 GB, 지연 민감 소규모	수백 GB 이상, 처리량 중심

인프라는 제품 이름부터 고르는 것이 아니라 워크로드 → 병목 → 자원 계층 → 운영 제약 순서로 결정한다. Volume이 크더라도 한 번만 순차 처리한다면 고속 로컬 SSD가 효율적일 수 있고, 데이터가 작더라도 초당 이벤트가 계속 유입되면 네트워크와 스트리밍 수집 계층이 먼저 병목이 될 수 있다.

표 2-3. 요구조건 기반 인프라 선택 매트릭스

관측된 요구조건	우선 점검 자원	1차 선택	다음 주차에서 검증할 내용
데이터가 단일 서버 메모리에 들어가고 지연이 중요함	CPU·메모리·로컬 NVMe	단일 노드 Scale-up	11주차 Spark 단일·분산 실행 비교
누적 데이터가 한 노드 저장 한계를 지속적으로 넘음	저장장치·네트워크	HDFS 또는 객체 저장소 기반 Scale-out	3~4주차 블록·복제·저장 포맷
이벤트가 중단 없이 유입되고 재처리가 필요함	네트워크·수집 버퍼	Kafka 로그와 소비자 그룹	9주차 partition·offset·replay
검색과 대시보드 응답이 빨라야 함	색인·메모리·샤드	Elasticsearch 수집·색인 계층	10주차 매핑·샤드·품질 경보
여러 저장소를 분석·운영 계층에 연결해야 함	메타데이터·권한·계보	데이터 레이크 또는 레이크하우스	15주차 통합·거버넌스·보안

CPU는 연산 병렬성, 메모리는 작업 집합과 셔플 버퍼, 저장장치는 순차·임의 I/O, 네트워크는 노드 간 셔플과 복제의 상한을 결정한다. GPU·TPU 같은 가속기는 반복적인 행렬 연산에 유리하지만 데이터 적재가 느리면 유휴 상태가 되므로, 가속기 도입 전에 저장장치와 네트워크 경로를 함께 측정해야 한다. 클라우드는 이 자원들을 사용량에 따라 조정하게 해 주고, 엣지는 원천 가까이에서 필터링해 지연과 전송량을 줄인다.

〔읽기 자료〕 이 주차의 다운로드 PDF 뒤에는 「빅데이터 인프라」 26쪽 원문이 부록으로 이어진다. 본문은 의사결정에 필요한 핵심만 다루며, 수집·저장·품질·거버넌스 상세 내용은 3·4·9·10·12·15주차에서 실습과 함께 다시 사용한다.

그림 2-2. 스케일업 대 스케일아웃 비교 도식 — 좌측은 단일 노드의 사양 상향, 우측은 동일 사양 노드의 수평 증설을 나타내며 하단 띠는 전환 임계점을 표시한다.

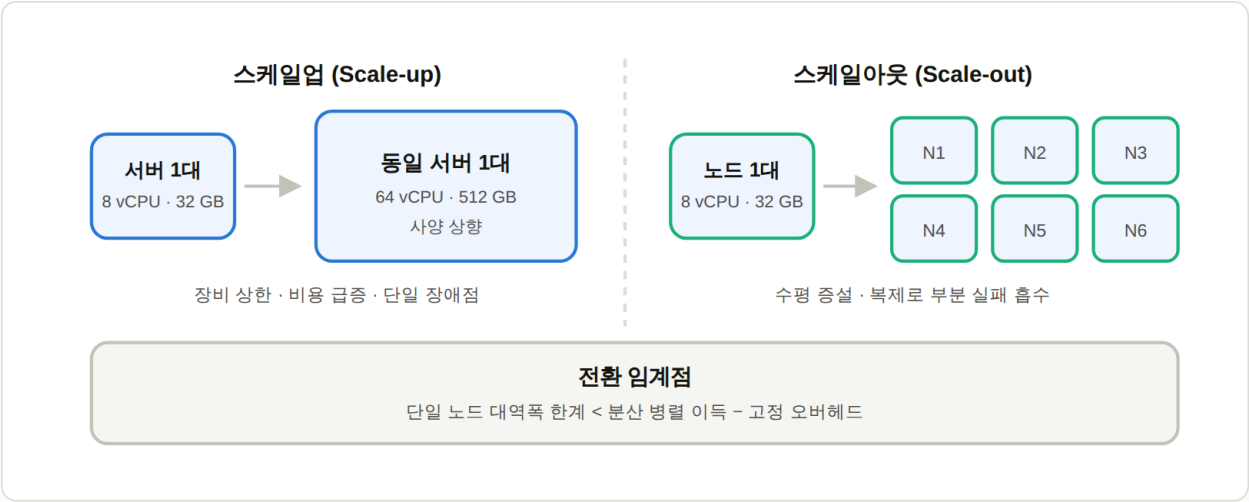


그림 2-2. 스케일업 대 스케일아웃 비교 도식

전환 임계점은 이론이 아니라 측정으로 판정한다. 분산 처리에는 작업 제출, 실행자(executor) 기동, 셔플 등 고정 오버헤드가 존재하므로 데이터가 작을 때는 오히려 단일 노드가 빠르다. 아래 예시 측정값은 8노드 클러스터를 기준으로 한 것이다.

표 2-4. 데이터 크기별 처리 시간 예시 측정값 (단위: 분)

데이터 크기	단일 노드	8노드 분산	비율
1 GB	1	1	1.0배
10 GB	9	2	4.5배
100 GB	88	13	6.8배
500 GB	440	58	7.6배
1 TB	900	116	7.8배

그림 2-3. 단일 노드 대 8노드 분산 처리 시간 비교 — 1 GB 구간에서는 두 방식의 차이가 없으나 10 GB를 넘어서면서 분산 처리의 이득이 나타나고, 100 GB 이상에서는 6배 이상으로 벌어진다. 값은 강의 실습 환경을 가정한 예시 수치이며 실제 값은 파일 포맷·압축·셔플량에 따라 달라진다.

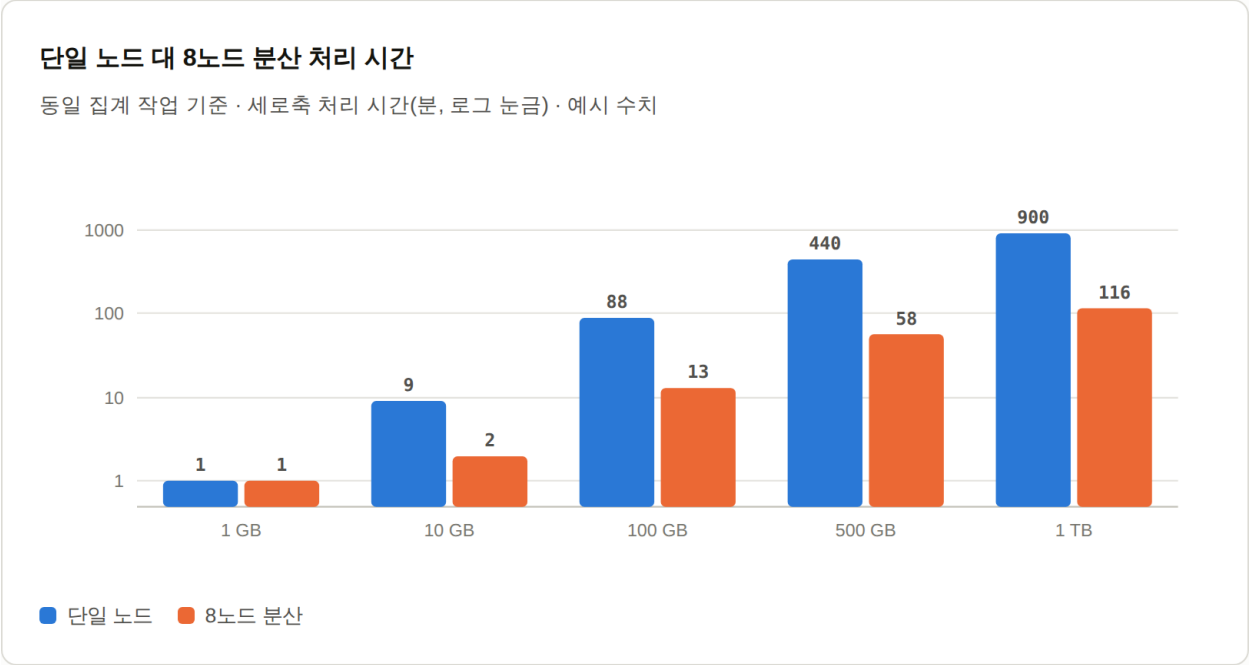


그림 2-3. 단일 노드 대 8노드 분산 처리 시간 비교

〔프로젝트 연결〕 팀 프로젝트 제안서에는 대상 데이터의 실측 용량과 일일 증가량을 반드시 기재한다. 규모 근거 없이 "빅데이터 처리를 위해 Spark를 사용한다"고 기술한 제안서는 반려 대상이다.

동일한 집계를 단일 노드와 분산 환경에서 각각 구현하면 임계점을 직접 확인할 수 있다.

```
import time
import pandas as pd

t0 = time.perf_counter()
df = pd.read_csv("/data/logs/access.csv")
top = df.groupby("status").size().sort_values(ascending=False).head(10)
print(top)
print("경과:", round(time.perf_counter() - t0, 2), "초")

from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
         .appName("ch02-status-count")
         .config("spark.sql.shuffle.partitions", "16")
         .getOrCreate())

df = spark.read.option("header", True).csv("hdfs:///data/logs/*.csv")
(df.groupBy("status")
   .agg(F.count("*").alias("cnt"))
   .orderBy(F.desc("cnt"))
   .show(10, truncate=False))

spark.stop()
```

코드 2-2. 동일 집계의 단일 노드 구현(pandas)과 분산 구현(PySpark 3.5.x). 두 결과가 일치하는지 먼저 확인한 뒤 시간을 비교한다.

```
# 처리 시간 측정: 3회 반복 후 중앙값을 사용한다
for i in 1 2 3; do
  /usr/bin/time -f "run${i} %e sec" python3 single_node.py
```

```
done

spark-submit --master local[8] spark_job.py 2>&1 | tail -5
```

코드 2-3. 측정 절차. 1회 측정은 캐시 상태에 따라 편차가 크므로 반복 측정한다.

2.3 빅데이터 생태계 지도

빅데이터 도구는 개별 제품이 아니라 파이프라인의 계층으로 이해해야 한다. 수집, 저장, 처리, 분석, 운영의 다섯 계층은 상위 계층이 하위 계층의 출력을 입력으로 받는 구조이다.

수집 계층은 외부 소스의 이벤트를 파이프라인 내부로 들여온다. Apache Kafka는 로그 기반 메시징 시스템으로 생산자와 소비자를 분리하고 재처리를 가능하게 한다. Filebeat와 Logstash는 파일·시스템 로그를 수집해 후속 계층에 전달한다.

저장 계층은 대용량 데이터를 분할·복제하여 보관한다. HDFS는 파일을 블록 단위로 나누어 여러 데이터노드(DataNode)에 복제 저장하며, 클라우드 환경에서는 Amazon S3와 같은 객체 스토리지가 같은 역할을 수행한다.

처리 계층은 저장된 데이터를 변환·집계한다. MapReduce는 디스크 기반 배치 처리 모델이고, Apache Spark는 인메모리 실행과 DAG 최적화로 반복 연산에서 우위를 갖는다.

분석 계층은 처리 결과에서 지표와 모델을 산출한다. Spark MLlib은 분산 환경에서 학습 파이프라인을 구성하고, Elasticsearch와 Kibana는 역색인 기반 검색과 대시보드를 제공한다.

운영 계층은 위 네 계층을 재현 가능하게 배포·운용한다. Docker는 실행 환경을 이미지로 고정하고, Kubernetes는 다수 컨테이너의 배치와 스케일링을 담당하며, CI/CD는 변경을 자동으로 검증·배포한다.

그림 2-4. 빅데이터 생태계 계층 지도 — 수집·저장·처리·분석·운영 다섯 계층과 각 계층의 대표 도구를 배치하고, 계층 간 데이터 흐름 방향을 표시한다.

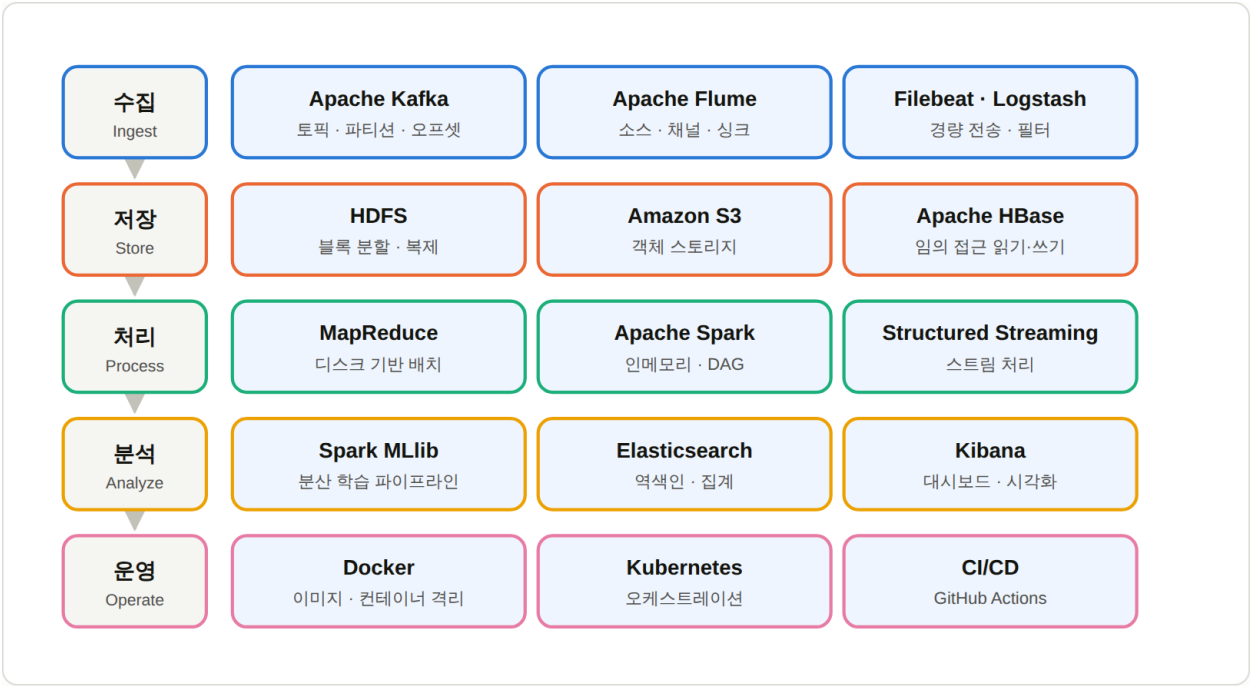


그림 2-4. 빅데이터 생태계 계층 지도

2.3.1 빅데이터 스택의 출발점: 시스템 소프트웨어

첨부 교안은 빅데이터 스택을 리눅스 운영체제 → Hadoop 분산 파일시스템 → MapReduce 병렬 처리 → Mahout 또는 Spark MLlib 분석의 네 계층으로 제시한다. 이 구조의 핵심은 상위 라이브러리만 아는 것이 아니라, 그 아래의 CPU·메모리·저장장치를 운영체제가 어떻게 관리하는지를 함께 이해하는 데 있다.

그림 2-5. 빅데이터 분석을 위한 소프트웨어 스택 — 첨부 교안 4쪽의 계층을 재구성한 것으로, 각 계층이 바로 아래 계층의 자원과 인터페이스를 사용한다.

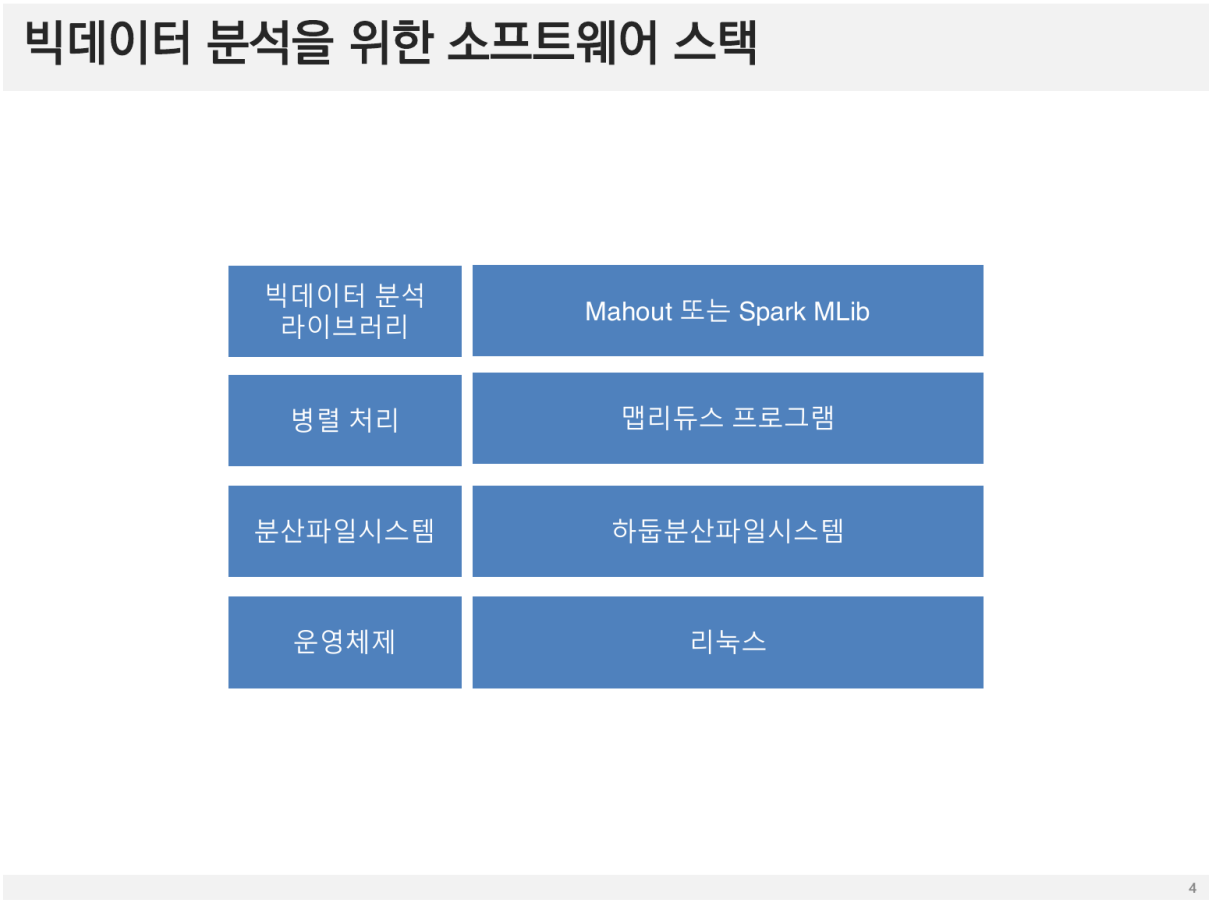


그림 2-5. 빅데이터 분석을 위한 소프트웨어 스택

시스템 소프트웨어는 하드웨어와 응용 소프트웨어 사이에서 자원을 배치하고 보호한다. 운영체제, 컴파일러·인터프리터, 데이터베이스, 장치 드라이버와 유틸리티가 이 범주에 속한다. 분산 처리 프로그램은 운영체제가 제공하는 프로세스, 가상 메모리, 파일과 네트워크 인터페이스 위에서만 실행된다.

2.3.2 커널·시스템 호출·드라이버

운영체제는 편리한 사용자 인터페이스를 제공하면서 제한된 컴퓨터 자원을 배분하고, 서로 다른 프로그램이 다른 작업 영역을 침범하지 못하게 보호한다. 핵심 구성은 다음과 같다.

구성요소	역할	빅데이터 실행에서의 의미
커널(Kernel)	CPU, 메모리, 저장장치 관리	executor와 daemon의 스케줄링·자원 격리
시스템 호출(System Call)	응용의 간접적 하드웨어 접근	파일 열기·읽기·쓰기·소켓 통신의 안전한 경계
드라이버(Driver)	장치별 제어 인터페이스	디스크·NIC·GPU 성능을 표준 인터페이스로 노출

그림 2-6. 운영체제의 계층 구조 — 응용 프로그램은 시스템 호출을 통해 커널에 작업을 요청하고, 커널은 드라이버를 통해 하드웨어와 상호작용한다.

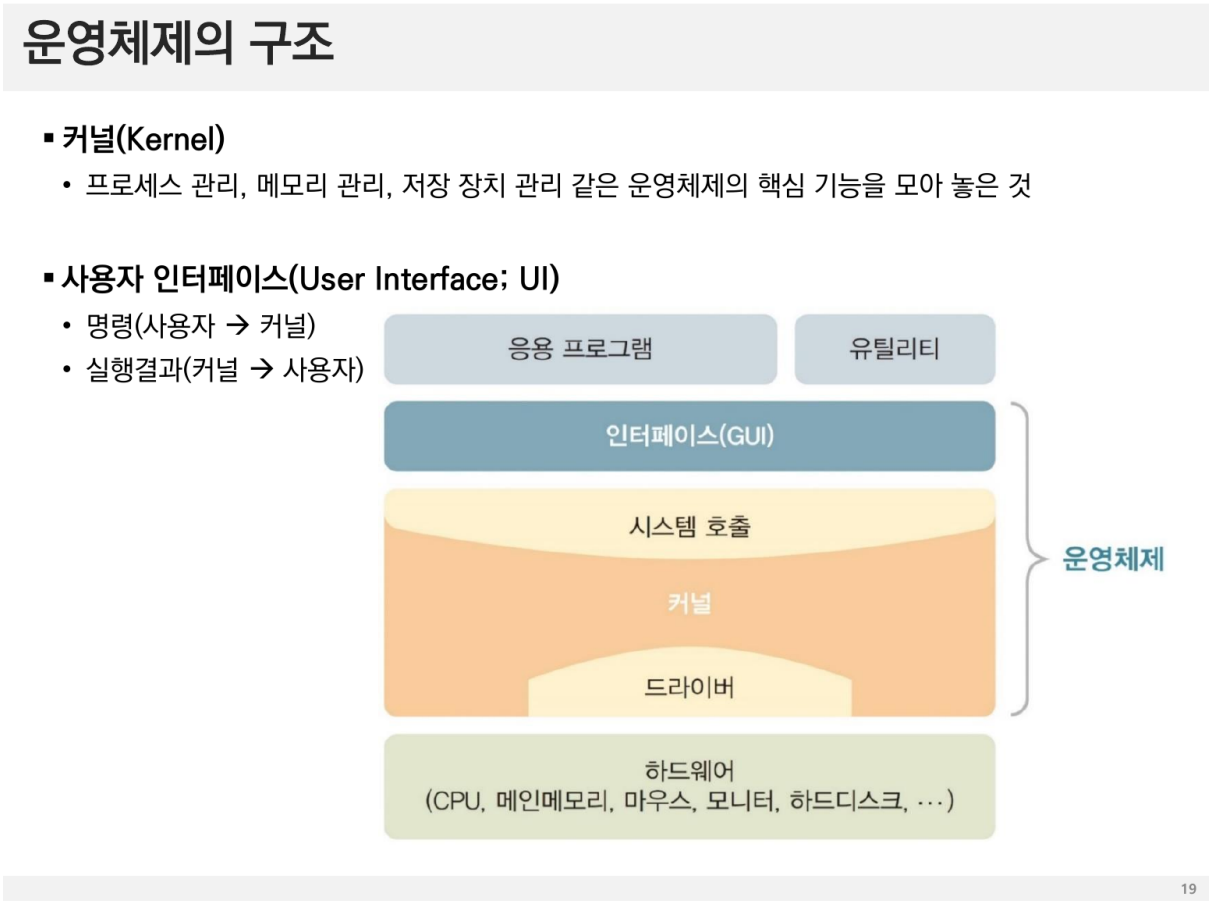


그림 2-6. 운영체제의 계층 구조

〔현장 연결〕 Spark 작업의 저장 속도가 느릴 때 DataFrame 코드만 보면 원인을 놓칠 수 있다. 파일시스템 캐시, 블록 크기, 디스크 I/O, 네트워크 드라이버와 커널 스케줄링까지 계층별로 확인해야 한다.

2.3.3 프로세스와 CPU 시분할

프로그램은 저장장치에 있는 정적인 코드이고, 프로세스(process)는 그 코드가 메모리에 적재되어 실행 중인 동적인 상태이다. 운영체제는 각 프로세스에 프로세스 제어 블록(PCB)을 부여하고 CPU 시간을 나누어 준다.

프로세스는 일반적으로 생성 → 준비 → 실행 → 종료를 거친다. CPU를 얻지 못한 작업은 준비 큐에서 기다리고, 할당 시간을 다 사용하면 다시 준비 상태로 돌아간다. Hadoop의 mapper·reducer, Spark의 driver·executor도 각각 이러한 프로세스로 실행되므로, 스케줄링 지연과 CPU 경쟁이 작업 성능에 직접 영향을 준다.

그림 2-7. 프로세스 상태 전이 — 생성·준비·실행·종료 상태와 CPU 할당 관계를 보여준다.

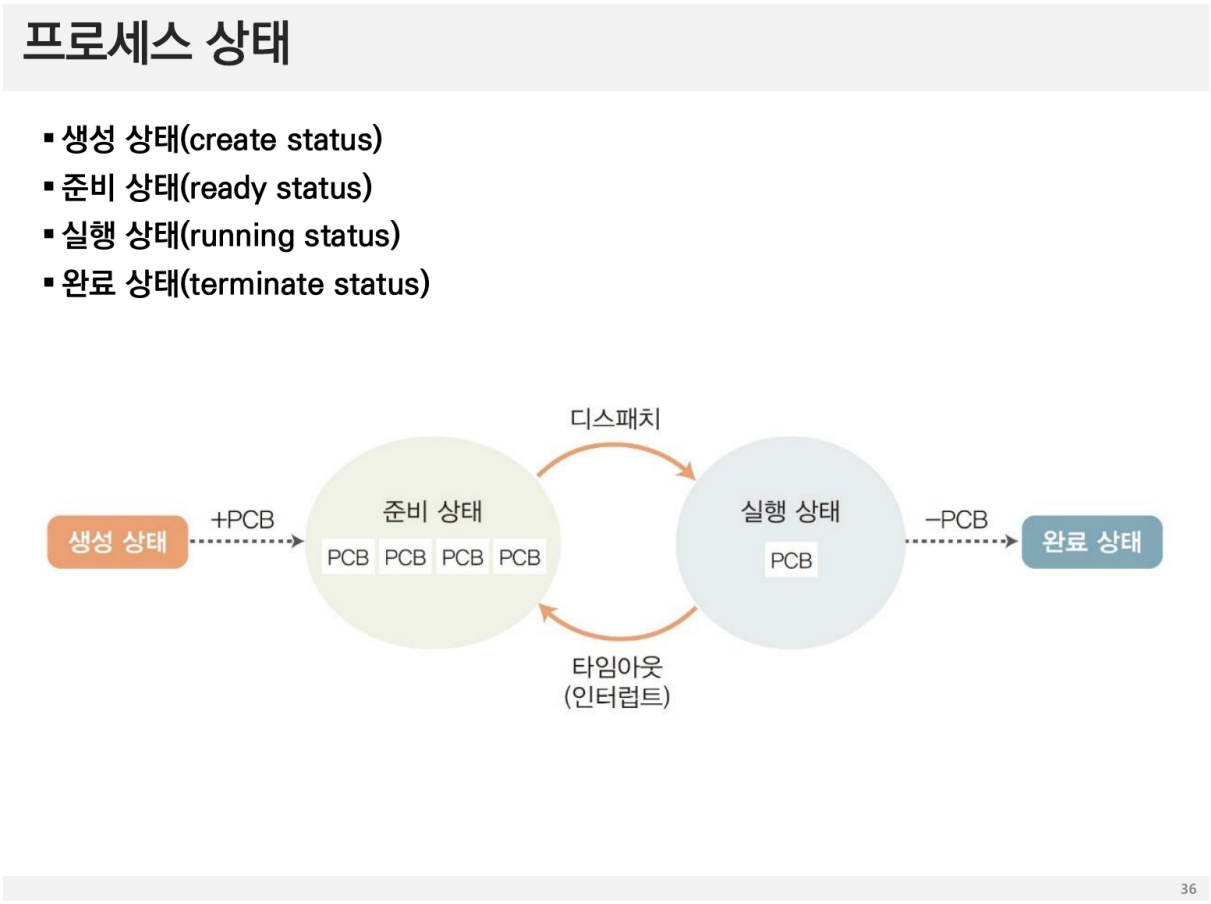


그림 2-7. 프로세스 상태 전이

```
# 현재 프로세스와 자원 사용량 확인
ps -eo pid,ppid,stat,%cpu,%mem,comm --sort=-%cpu | head
free -h
df -hT
```

코드 2-4. 리눅스에서 프로세스·메모리·파일시스템을 함께 점검하는 최소 명령 목록.

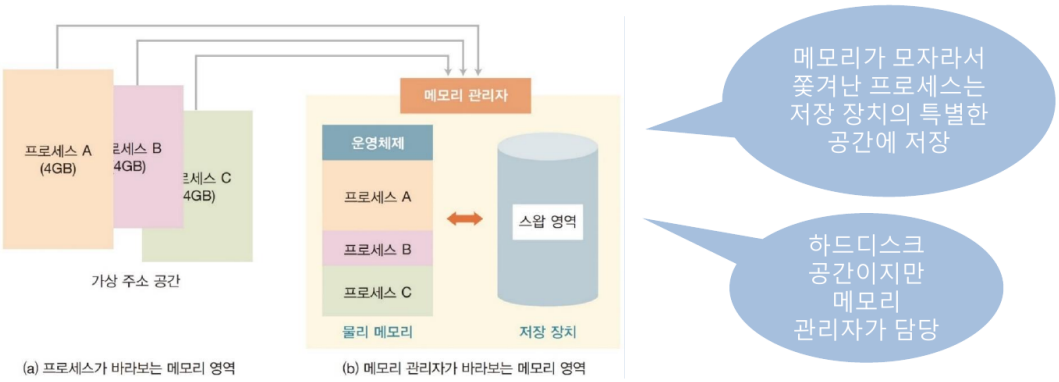
2.3.4 메모리 관리와 가상 메모리

모든 프로그램은 실행되려면 메모리에 올라와야 한다. 메모리 관리자는 프로세스와 데이터를 가져오고(fetch), 적절한 영역에 배치하며(placement), 공간이 부족하면 기존 페이지를 저장장치로 재배치한다(replacement). 가상 메모리는 RAM과 스왑 영역을 조합해 프로세스에게 더 큰 논리 주소 공간을 제공한다.

그림 2-8. 가상 메모리와 스왑 영역 — 물리 메모리보다 큰 프로그램을 실행할 때 일부 페이지가 저장장치로 이동하는 구조를 보여준다.

가상 메모리 (virtual memory)

- **메인메모리(1GB ~ 16GB)일 때, 32GB 프로그램을 어떻게 실행시킬까?**
- **실제 메모리 크기와 상관없이 프로세스에 커다란 메모리 공간을 제공하는 기술**



47

그림 2-8. 가상 메모리와 스왑 영역

Spark에서 스필(spill)이 발생한다는 것은 중간 데이터가 실행 메모리에 다 들어가지 못해 디스크로 밀려났다는 뜻이다. 운영체제의 스왑과 Spark의 스필은 서로 다른 계층의 현상이지만, 둘 다 지속되면 저장장치 I/O가 급증해 처리 시간이 길어진다.

2.3.5 파일시스템·블록·빈 공간

파일은 파일명·크기·생성 시간 등의 메타데이터와 실제 내용으로 구성된 논리적 데이터 집합이다. 디렉터리는 관련 파일을 계층적으로 조직하며, 파일시스템은 파일명과 저장 위치를 파일 테이블로 관리한다. Windows의 FAT·NTFS와 Unix/Linux의 inode는 이 메타데이터를 관리하는 방식의 예이다.

운영체제와 저장장치 사이의 기본 전송 단위는 블록(block)이다. 작은 블록은 공간 낭비를 줄이지만 많은 I/O 요청을 발생시키고, 큰 블록은 대형 파일을 연속적으로 처리하기에 유리하지만 작은 파일의 내부 낭비가 커질 수 있다. HDFS가 128 MB 등 큰 블록을 사용하는 이유도 대용량 순차 처리에서 탐색·메타데이터 오버헤드를 줄이기 위해서다.

그림 2-9. 저장장치의 블록 단위 전송 — 섹터를 묶은 블록이 운영체제와 저장장치 사이의 전송 단위가 되는 구조다.

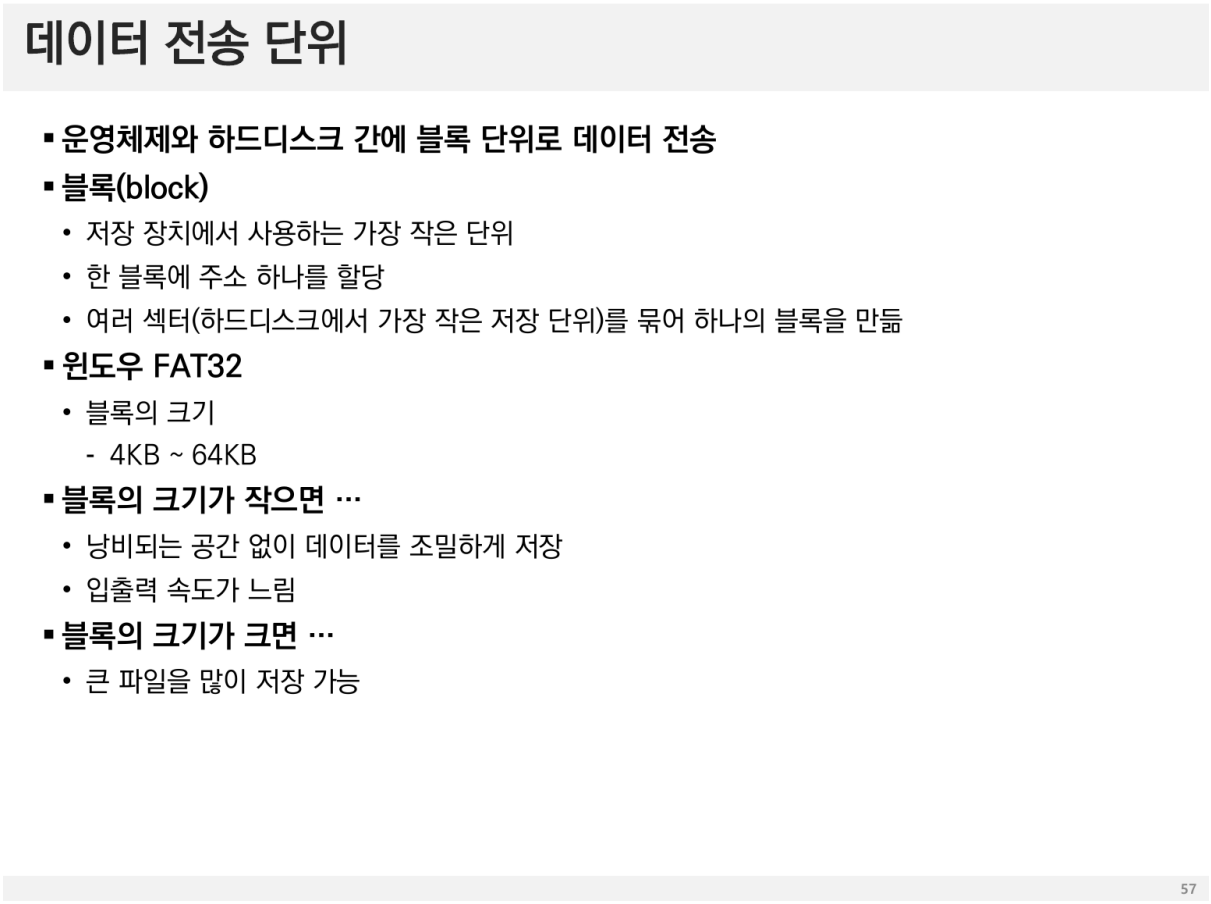


그림 2-9. 저장장치의 블록 단위 전송

파일을 삭제해도 데이터 비트가 즉시 사라지는 것은 아니다. 파일 테이블의 항목을 제거하고 사용하던 블록을 빈 공간 목록에 등록한 뒤, 다른 데이터가 덮어쓸 때 실제 내용이 사라진다. 이 원리는 데이터 복구·포렌식과 안전한 파기 정책을 이해하는 기초가 된다.

〔실습 연결〕 3·4주차 HDFS 실습에서는 로컬 파일시스템의 블록과 HDFS 블록을 구분하고, 파일 크기·블록 수·복제 계수가 저장 공간과 NameNode 메타데이터에 미치는 영향을 측정한다.

〔더 알아보기〕 각 도구의 현행 안정 버전과 호환 행렬은 공식 문서에서 확인한다. Hadoop 3.4.x는 hadoop.apache.org/docs/stable, Spark 3.5.x는 spark.apache.org/docs/latest, Kafka는 kafka.apache.org/documentation, Elastic Stack 8.x는 elastic.co/guide에 각각 게시되어 있다. 커넥터 좌표는 반드시 런타임 버전과 대조한다.

2.4 생성형 AI의 개발 보조 역할과 한계

생성형 AI는 파이프라인 설계 초안 작성, 상용구 코드 생성, 오류 메시지 해석, 문서 초안 작성에서 유효한 생산성 향상을 제공한다. 반면 존재하지 않는 API의 생성, 버전 간 호환성 오판, 근거 없는 성능 단언은 반복적으로 관찰되는 실패 유형이다. 학습 데이터의 시점이 고정되어 있으므로 최근 릴리스의 변경 사항이 반영되지 않는 경우가 많다.

따라서 본 교과목은 생성형 AI를 코드 생성기가 아니라 검증 대상이 있는 초안 생성기로 다룬다. 산출물은 실행 검증, 버전 대조, 근거 확인을 통과한 뒤에만 저장소에 반영한다.

표 2-5. 생성형 AI 활용 가능 작업과 검증 필수 작업

구분	작업	판단 근거	처리 절차
활용 가능	파이프라인 계층 구조 초안	널리 공개된 아키텍처 패턴	검토 후 채택
활용 가능	설정 파일·Dockerfile 상용구	반복적 구조, 오류가 즉시 드러남	빌드 실행으로 확인
활용 가능	오류 메시지 해석·용어 정리	검색 가능한 공개 지식	공식 문서 대조
검증 필수	API 시그니처·함수명	존재하지 않는 심볼 생성 빈발	dir()·공식 API 문서 확인
검증 필수	라이브러리·커넥터 버전 좌표	학습 시점 이후 릴리스 미반영	저장소 좌표·릴리스 노트 대조
검증 필수	성능·처리량 수치	측정 근거 없이 단언하는 경향	실측 후 수치 대체
검증 필수	보안 설정·인증 구성	예제 수준의 취약 설정 제시	보안 가이드 기준 재작성

〔AI 함정〕 생성형 AI는 pyspark.sql.functions.auto_schema처럼 그럴듯하지만 존재하지 않는 함수를 자연스럽게 제시한다. 함수명이 낯설면 임포트 성공 여부부터 확인한다. python3 -c "from pyspark.sql import functions as F; print(hasattr(F, 'auto_schema'))"가 False를 반환하면 해당 심볼은 존재하지 않는다.

3. 실습

STEP 1. 요구사항 정의 대상 데이터, 일일 증가량, 요구 지연, 산출물을 문장으로 확정한다. 예시 요구사항은 "웹 접근 로그 일 12 GB, 종단 지연 5분 이내, 상태코드별 5분 단위 집계 대시보드"이다. 요구사항이 확정되지 않은 상태의 프롬프트는 검증 기준을 만들 수 없다.

STEP 2. 구조화 프롬프트 작성 프롬프트를 자연어 문단이 아니라 명세 파일로 관리한다. 역할, 입력 조건, 제약, 산출 형식, 검증 기준을 분리해 기술하면 동일 프롬프트를 재사용하고 결과를 비교할 수 있다.

```
role: 빅데이터 파이프라인 아키텍트
task: 웹 접근 로그 실시간 집계 파이프라인 설계 초안 작성
input:
  source: 웹 서버 access log
  daily_volume_gb: 12
  latency_requirement: 종단 5분 이내
constraints:
  runtime:
    hadoop: 3.4.x
    spark: 3.5.x
    kafka: 3.x
    elasticsearch: 8.x
  environment: Docker Compose 단일 호스트, 8 vCPU / 32 GB
  forbidden: 유료 관리형 서비스 사용 금지
output_format:
  - 계층별 구성요소 표
  - PySpark 예제 코드
  - 의존성 좌표 목록(groupId:artifactId:version)
verification:
  - 모든 함수는 공식 API 문서에 존재해야 한다
  - 모든 의존성 좌표는 런타임 버전과 일치해야 한다
```

```
- 성능 수치는 제시하지 않고 측정 항목만 제시한다
```

코드 2-5. 구조화 프롬프트 명세. verification 절이 이후 결함 판정의 기준이 된다.

STEP 3. 설계 초안 수집 STEP 2의 명세를 생성형 AI에 입력하고 산출물을 원문 그대로 저장한다. 원문을 수정하지 않고 보존해야 결함 탐지 결과를 재현할 수 있다. 아래는 검증 전 초안의 예시이다.

```
# 생성형 AI 초안 (검증 전 원문, 결함 포함)
from pyspark.sql import SparkSession
from pyspark.sql.functions import auto_schema          # (A)
from kafka import KafkaStreamReader                    # (B)

spark = (SparkSession.builder
        .config("spark.jars.packages",
                "org.apache.spark:spark-sql-kafka-0-10_2.12:2.4.0")  # (C)
        .getOrCreate())

df = (spark.readStream.format("kafka")
      .option("kafka.bootstrap.servers", "localhost:9092")
      .option("subscribe", "access-log")
      .load())

top = df.groupBy("value").count().orderBy("count", ascending=False)   # (D)
top.writeStream.format("console").start().awaitTermination()
# 주석: "1 TB를 8노드에서 약 30초에 처리한다"                          # (E)
```

코드 2-6. 검증 대상 초안. 결함 표시는 STEP 4에서 부여한 식별자이다.

STEP 4. 결함 탐지 명세의 verification 항목별로 기계적으로 확인한다. 판단이 아니라 명령 실행 결과를 근거로 삼는다.

```
# (A) 존재하지 않는 함수 확인
python3 -c "from pyspark.sql import functions as F; print(hasattr(F, 'auto_schema'))"

# (B) 존재하지 않는 클래스 확인
python3 -c "import kafka; print([n for n in dir(kafka) if 'Stream' in n])"

# (C) 런타임 버전과 커넥터 좌표 대조
spark-submit --version
pip index versions pyspark

# (D) 스트리밍 제약 확인: 집계 후 정렬은 complete 모드에서만 허용된다
spark-submit --master 'local[4]' draft_pipeline.py 2>&1 | grep -i 'AnalysisException'

# (E) 성능 단언 검증: 근거 측정치가 없으면 문장을 삭제한다
```

코드 2-7. 결함 검증 명령. (A)와 (B)가 각각 False, 빈 목록을 반환하면 해당 심볼은 존재하지 않는다.

STEP 5. 검증 리포트 작성 결함을 기계 판독 가능한 형식으로 기록한다. 결함 유형, 근거, 수정안을 분리해 기재해야 이후 과제3에서 동일 형식으로 누적할 수 있다.

```
{
  "review_id": "ch02-ai-review-01",
  "prompt_spec": "prompt/pipeline_draft.yaml",
  "defects": [
    {"id": "A", "type": "nonexistent_api",
     "evidence": "hasattr(F,'auto_schema') == False",
     "fix": "spark.read.option('inferSchema', True) 또는 명시적 StructType 사용"},
    {"id": "B", "type": "nonexistent_class",
     "evidence": "kafka 패키지에 KafkaStreamReader 없음",
     "fix": "spark.readStream.format('kafka') 사용, 별도 클라이언트 불필요"},
    {"id": "C", "type": "version_mismatch",
     "evidence": "런타임 Spark 3.5.x, 좌표는 2.4.0",
```

```
    "fix": "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3"},
   {"id": "D", "type": "unsupported_operation",
    "evidence": "스트리밍 집계 정렬은 complete 출력 모드에서만 허용",
    "fix": "outputMode('complete') 지정 또는 정렬 제거"},
   {"id": "E", "type": "unfounded_performance_claim",
    "evidence": "측정 근거 없음",
    "fix": "성능 문장 삭제 후 실측치로 대체"}
  ],
  "verdict": "reject_with_revision"
}
```

코드 2-8. 검증 리포트. 저장소의 docs/ai-review/ 경로에 커밋하여 이력을 남긴다.

STEP 6. 수정안 재요청과 재검증 결함 목록을 프롬프트에 추가 제약으로 반영하여 재요청한다. 재요청 후에도 STEP 4의 명령을 동일하게 재실행하고, 통과한 항목만 초안을 갱신한다. 재검증 없이 채택하는 절차는 허용하지 않는다.

> 〔프로젝트 연결〕 이 실습의 검증 리포트 형식은 13주차 과제3(생성형 AI 활용 데이터 분석·문서화)의 제출 형식과 동일하다. 2주차부터 동일 형식으로 축적하면 과제3에서 재작성이 필요 없다.

---

4. 요약

- 5V — Volume·Velocity·Variety·Veracity·Value는 데이터의 성격을 진단하는 다섯 축이며, 축마다 측정지표와 대응 기술 계층이 대응된다.
- 규모의 임계점 — 빅데이터 여부는 용량 절대값이 아니라 단일 노드의 처리 한계를 넘는지로 판정한다.
- 스케일업 — 코드 변경이 없는 대신 사양 상한과 비용 급증, 단일 장애점이 남는다.
- 스케일아웃 — 상한이 사실상 없으나 분산 실행 모델 재작성과 셔플·부분 실패 처리를 요구한다.
- 고정 오버헤드 — 분산 처리는 수십 초 단위 고정 비용이 있어 소규모 데이터에서는 단일 노드가 유리하다.
- 생태계 5계층 — 수집(Kafka·Beats), 저장(HDFS·S3), 처리(MapReduce·Spark), 분석(Spark MLlib·Elastic Stack), 운영(Docker·Kubernetes·CI/CD)으로 구조화된다.
- 운영체제 기반 — 커널·시스템 호출·드라이버가 분산 응용과 CPU·메모리·저장장치 사이의 실행 경계를 구성한다.
- 프로세스와 메모리 — 프로세스 상태 전이, CPU 시분할, 가상 메모리와 스왑은 executor 스케줄링과 스필 성능을 이해하는 기초다.
- 파일시스템과 블록 — 파일 메타데이터, 블록 크기, 빈 공간 관리는 HDFS의 블록·복제 설계로 이어진다.
- AI 검증 의무 — 존재하지 않는 API, 버전 불일치, 근거 없는 성능 단언은 명령 실행 결과로 판정하고 리포트에 기록한다.
- 다음 주차 예고 — 3주차에서는 저장 계층의 HDFS 아키텍처를 다루고 컨테이너 기반 Hadoop 클러스터를 직접 구축한다.

---

5. 퀴즈

1. Velocity의 직접 지표는?

1) events/sec와 p95 지연

2) 파일 확장자

3) CPU 브랜드

4) 그래프 색

정답: 1 해설: 유입률과 처리 지연·backlog가 속도 압력을 나타낸다.

2. Scale-out의 고유한 고려사항은?

1) 키보드 배열

2) 네트워크 셔플과 노드 장애

3) 글꼴

4) 파일명 길이

정답: 2 해설: 분산 노드 간 통신과 부분 장애를 설계해야 한다.

3. Value를 올바르게 판단하는 방법은?

1) 용량만 측정

2) AI 답을 채택

3) KPI·오판 비용·인프라 비용 비교

4) 행 수가 크면 자동 인정

정답: 3 해설: 활용 효용은 의사결정 맥락과 비용을 포함한다.

4. AI가 제시한 낯선 API의 첫 검증은?

1) 발표 자료에 복사

2) 이름을 줄임

3) 경고 억제

4) import와 공식 버전 문서 확인

정답: 4 해설: 심볼의 실재와 현재 버전 호환성을 먼저 확인한다.

3주차 (인프라)

Chapter 2. 빅데이터 인프라와 수집·저장 기술
- 학습 목표
- 2.1 빅데이터 시대의 하드웨어 인프라 혁명
  - 전통적 컴퓨팅 환경의 한계
  - 분산 컴퓨팅의 등장과 필요성
  - 하드웨어 아키텍처의 구체적 변화
  - 클라우드 컴퓨팅의 영향
  - 특수 목적 하드웨어의 등장
  - 엣지 컴퓨팅의 새로운 패러다임
- 2.2 빅데이터 수집 기술
  - 웹 크롤링과 스크래핑 기술
  - API를 통한 데이터 연동
  - 센서 데이터 수집과 IoT
  - 데이터베이스 연동과 ETL
  - 스트리밍 데이터 수집
- 2.3 빅데이터 저장 시스템
  - 분산 파일 시스템: HDFS와 그 진화
  - NoSQL 데이터베이스의 다양한 모델
  - 클라우드 스토리지와 데이터 레이크
- 2.4 데이터 품질 관리
  - 데이터 정제와 전처리
  - 데이터 검증과 모니터링
  - 데이터 표준화와 메타데이터 관리
- 2.5 데이터 거버넌스와 보안
  - 개인정보보호와 규제 준수
  - 데이터 보안과 접근 제어

Chapter 2. 빅데이터 인프라와 수집·저장 기술

학습 목표

💡 이 장을 통해 독자는 빅데이터 처리를 위한 하드웨어 인프라의 근본적인 변화를 이해하고, 전통적인 중앙 집중형 시스템에서 분산 컴퓨팅 환경으로의 전환 과정과 그 필요성을 학습하게 될 것입니다. 또한 다양한 데이터 수집 기술의 원리와 특성을 파악하고, 빅데이터 저장을 위한 새로운 시스템들의 구조와 작동 방식을 깊이 있게 탐구할 것입니다. 마지막으로 데이터 품질 관리와 거버넌스의 중요성을 이해하고, 실무에서 적용할 수 있는 구체적인 방법론을 습득할 수 있을 것입니다.

2.1 빅데이터 시대의 하드웨어 인프라 혁명

전통적 컴퓨팅 환경의 한계

- 빅데이터 시대 이전의 기업 IT 인프라는 주로 중앙집중형(Centralized) 아키텍처를 기반으로 구축되었습니다.
  이러한 환경에서는 하나의 강력한 서버나 메인프레임이 모든 데이터 처리와 연산을 담당했으며,
  성능 향상을 위해서는 더 빠른 CPU, 더 많은 메모리, 더 큰 저장 공간을 가진 고성능 하드웨어로 업그레이드하는 수직적 확장(Scale-up) 방식을 사용했습니다.
- 1990년대와 2000년대 초반의 전형적인 기업 데이터센터를 살펴보면, 몇 대의 고성능 서버가 모든 업무를 처리하는 구조였습니다.
  오라클 Exadata나 IBM Power Systems와 같은 고성능 서버들이 수십만 달러에서 수백만 달러의 비용으로 도입되었으며, 이들은 단일 노드에서 최대한의 성능을 발휘하도록 설계되었습니다.
- 이러한 중앙집중형 시스템은 관리가 용이하고 데이터 일관성 유지가 쉬웠지만, 몇 가지 근본적인 한계를 가지고 있었습니다.
  - 첫째, 하드웨어의 물리적 한계로 인해 성능 향상에는 분명한 천장이 존재했습니다. 아무리 고성능 CPU와 대용량 메모리를 탑재해도 단일 서버가 처리할 수 있는 데이터량과 연산량에는 한계가 있었습니다.
  - 둘째, 비용 효율성의 문제가 있었습니다. 성능을 두 배로 늘리려면 하드웨어 비용이 네 배 이상 증가하는 경우가 많았으며, 이는 기업의 IT 예산에 큰 부담을 주었습니다.
  - 셋째, 단일 장애점(Single Point of Failure)의 문제가 심각했습니다. 중앙 서버에 장애가 발생하면 전체 시스템이 마비되는 위험이 있었습니다.
  - 넷째, 확장성의 한계가 명확했습니다. 데이터량이 기하급수적으로 증가하는 환경에서 수직적 확장만으로는 비용과 성능 측면에서 지속 가능한 해결책을 제공하기 어

려웠습니다.
특히 구글, 아마존, 페이스북과 같은 인터넷 기업들이 다루는 데이터 규모는 전통적인 시스템의 처리 능력을 훨씬 초과했습니다.

분산 컴퓨팅의 등장과 필요성

- 빅데이터 시대의 도래와 함께 전통적인 중앙집중형 시스템의 한계가 명확해지면서, 새로운 패러다임인 분산 컴퓨팅(Distributed Computing)이 주목받기 시작했습니다.
  분산 컴퓨팅은 여러 대의 상대적으로 저렴한 서버를 네트워크로 연결하여 하나의 큰 컴퓨터처럼 사용하는 수평적 확장(Scale-out) 방식입니다.
- 이러한 패러다임 전환의 배경에는 몇 가지 기술적, 경제적 동인이 있었습니다. 먼저 인터넷의 급속한 발전으로 네트워크 대역폭이 크게 향상되었습니다.
  기가비트 이더넷의 보급과 광통신 기술의 발전으로 서버 간 고속 통신이 가능해졌으며, 이는 분산 시스템의 네트워크 오버헤드를 크게 줄였습니다.
- 또한 상용 하드웨어(Commodity Hardware)의 성능이 급격히 향상되었습니다. 무어의 법칙에 따라 일반적인 x86 서버의 성능이 꾸준히 개선되었으며, 동시에 가격은 하락했습니다.
  이로 인해 여러 대의 일반 서버를 사용하는 것이 한 대의 고성능 서버를 사용하는 것보다 비용 효율적이 되었습니다.
- 구글의 사례는 이러한 전환의 대표적인 예시입니다. 구글은 2000년대 초부터 수만 대의 일반 PC급 서버를 클러스터로 구성하여 검색 엔진을 운영했습니다.
  이들 서버는 개별적으로는 높은 성능을 가지지 않았지만, 수천 대가 협력하여 작업할 때는 어떤 슈퍼컴퓨터보다도 강력한 성능을 발휘했습니다.
- 분산 컴퓨팅의 핵심 아이디어는 '장애를 전제로 한 설계'입니다. 수천 대의 서버로 구성된 클러스터에서는 항상 일부 서버에 장애가 발생한다고 가정하고, 이러한 장애가 전체 시스템의 동작에 영향을 주지 않도록 설계합니다. 이는 고가의 고성능 하드웨어에 의존하여 장애 확률을 최소화하려던 전통적 접근법과는 근본적으로 다른 철학입니다.

하드웨어 아키텍처의 구체적 변화

- **CPU 아키텍처의 변화**
  - 전통적인 시스템에서는 단일 CPU의 클럭 속도를 높이는 것이 주된 성능 향상 방법이었습니다. 하지만 물리적 한계와 전력 소비 문제로 인해 클럭 속도 향상이 둔화되면서, CPU 제조사들은 멀티코어 아키텍처로 전환했습니다. 인텔의 제온(Xeon) 시리즈나 AMD의 에픽(EPYC) 시리즈는 하나의 칩에 수십 개의 코어를 집적하여 병렬 처리 능력을 크게 향상시켰습니다.

    - 빅데이터 환경에서는 이러한 멀티코어 CPU를 여러 개 탑재한 서버들이 클러스터를 구성합니다. 예를 들어, 하둡 클러스터의 일반적인 노드는 2개의 8코어 CPU를 탑재하여 총 16개의 처리 코어를 가지며, 수백 대의 이러한 노드들이 수만 개의 코어를 제공하는 거대한 병렬 처리 시스템을 구성합니다.

- **메모리 시스템의 진화**
    - 전통적인 시스템에서는 하나의 서버에 최대한 많은 메모리를 탑재하는 것이 성능 향상의 핵심이었습니다. 하지만 빅데이터 환경에서는 각 노드가 적당한 양의 메모리를 가지고, 이를 클러스터 전체적으로 활용하는 분산 메모리 아키텍처가 주류가 되었습니다.
    - 인메모리 컴퓨팅 기술의 발전으로 Apache Spark와 같은 프레임워크는 클러스터 전체의 메모리를 하나의 큰 캐시처럼 활용할 수 있게 되었습니다. 이는 디스크 I/O를 최소화하여 데이터 처리 속도를 크게 향상시켰습니다.
    - 또한 메모리 기술 자체의 발전도 주목할 만합니다. DDR4에서 DDR5로의 전환, 인텔의 옵테인(Optane) 같은 비휘발성 메모리 기술의 등장은 메모리와 저장장치 간의 경계를 모호하게 만들고 있습니다.
- **저장장치의 혁명**
    - 빅데이터 시대에 가장 극적인 변화를 겪은 것은 저장장치입니다. 전통적인 시스템에서는 고속의 SCSI 하드드라이브나 SAN(Storage Area Network)을 사용했지만,
      빅데이터 환경에서는 각 서버에 로컬 저장장치를 탑재하고 이를 네트워크로 연결하는 분산 저장 시스템이 주류가 되었습니다.
    - HDFS(Hadoop Distributed File System)는 이러한 분산 저장의 대표적인 예시입니다. 각 노드의 일반적인 SATA 하드드라이브들을 네트워크로 연결하여 페타바이트급의 저장 공간을 제공하며, 데이터 복제를 통해 안정성을 확보합니다.
    - SSD의 급속한 가격 하락도 중요한 변화 요인입니다. 2010년경만 해도 SSD는 용량 대비 가격이 HDD의 10배 이상이었지만, 현재는 2-3배 수준으로 격차가 줄어들었습니다. 이에 따라 많은 빅데이터 시스템들이 HDD와 SSD를 계층적으로 활용하는 하이브리드 저장 시스템을 도입하고 있습니다.
    - NVMe(Non-Volatile Memory Express) SSD의 등장은 또 다른 혁신을 가져왔습니다. 기존 SATA 인터페이스의 속도 제한을 뛰어넘어 초당 수 GB의 데이터 전송 속도를 제공하며, 이는 실시간 빅데이터 분석을 가능하게 하는 핵심 기술 중 하나입니다.
- **네트워크 인프라의 발전**

- 분산 시스템에서 네트워크는 시스템의 성능을 결정하는 핵심 요소입니다. 전통적인 기업 환경에서는 100Mbps 이더넷이 일반적이었지만,
  빅데이터 클러스터에서는 10Gbps, 40Gbps, 심지어 100Gbps 네트워크가 사용됩니다.
- 인피니밴드(InfiniBand)와 같은 고성능 네트워크 기술은 초저지연(Ultra-low Latency) 통신을 제공하여 분산 시스템의 성능을 크게 향상시켰습니다.
  특히 고성능 컴퓨팅(HPC) 분야에서 검증된 이러한 기술들이 빅데이터 분야로 확산되고 있습니다.
- 소프트웨어 정의 네트워킹(SDN)의 도입으로 네트워크 설정과 관리도 훨씬 유연해졌습니다. 오픈플로우(OpenFlow) 프로토콜을 통해 네트워크 트래픽을 동적으로 조절하여 데이터 처리 작업에 최적화된 네트워크 경로를 제공할 수 있게 되었습니다.

클라우드 컴퓨팅의 영향

> 💡 클라우드 컴퓨팅의 성숙은 빅데이터 인프라에 또 다른 혁명을 가져왔습니다.

- Amazon Web Services(AWS), Google Cloud Platform(GCP), Microsoft Azure와 같은 클라우드 서비스 제공업체들은 필요에 따라 수천 대의 서버를 즉시 사용할 수 있는 인프라를 제공합니다.
- 클라우드 환경에서는 탄력적 확장(Elastic Scaling)이 가능합니다. 데이터 처리 작업의 규모에 따라 몇 분 안에 서버의 수를 늘리거나 줄일 수 있어, 자원 활용의 효율성을 크게 향상시켰습니다.
  이는 전통적인 **온프레미스(On-Premise) 환경**에서는 불가능했던 유연성입니다.
- 클라우드 제공업체들은 또한 관리형 빅데이터 서비스를 제공합니다. AWS의 EMR(Elastic MapReduce), 구글의 Dataproc, 마이크로소프트의 HDInsight 등은 복잡한 클러스터 설정과 관리를 자동화하여 사용자가 데이터 분석에만 집중할 수 있게 해줍니다.

컨테이너 기술의 부상

- 최근에는 **Docker와 Kubernetes** 같은 컨테이너 기술이 빅데이터 인프라에 새로운 변화를 가져오고 있습니다. 컨테이너는 가상머신보다 가벼우면서도 애플리케이션 간의 격리를 제공하여, 하나의 클러스터에서 다양한 빅데이터 워크로드를 효율적으로 실행할 수 있게 해줍니다.

- Kubernetes를 기반으로 한 빅데이터 플랫폼들이 등장하면서, 스파크, 하둡, 카프카 등 다양한 빅데이터 도구들을 컨테이너로 패키징하여 배포하고 관리하는 것이 가능해졌습니다. 이는 인프라 관리의 복잡성을 크게 줄이고 자원 활용률을 향상시켰습니다.

특수 목적 하드웨어의 등장

- 최근에는 AI와 머신러닝 워크로드의 증가로 인해 특수 목적 하드웨어가 주목받고 있습니다. NVIDIA의 GPU는 병렬 처리에 특화되어 딥러닝 학습과 추론에서 CPU보다 수십 배 빠른 성능을 제공합니다.
- 구글의 TPU(Tensor Processing Unit)는 텐서플로우 연산에 최적화된 전용 칩으로, 대규모 신경망 학습에서 기존 GPU보다도 월등한 성능을 보여줍니다. 아마존의 Inferentia, 인텔의 Nervana 같은 AI 전용 칩들도 각각 고유한 강점을 가지고 시장에 등장하고 있습니다.
- 이러한 특수 목적 하드웨어들은 전통적인 범용 CPU 중심의 아키텍처에서 벗어나 작업 특성에 맞는 최적화된 하드웨어를 선택하는 이기종 컴퓨팅(Heterogeneous Computing) 환경으로의 전환을 이끌고 있습니다.

엣지 컴퓨팅의 새로운 패러다임

- 5G 네트워크의 상용화와 IoT 기기의 폭발적 증가로 인해 엣지 컴퓨팅이 새로운 트렌드로 부상하고 있습니다. 모든 데이터를 중앙 데이터센터로 전송하여 처리하는 대신, 데이터 생성 지점 근처에서 1차 처리를 수행하는 분산 아키텍처입니다.
- 자율주행 자동차, 스마트 팩토리, 증강현실 애플리케이션 등은 밀리세컨드 단위의 실시간 응답이 필요하기 때문에, 클라우드까지 왕복하는 네트워크 지연 시간을 감수할 수 없습니다. 이러한 요구사항을 충족하기 위해 엣지 데이터센터, 기지국 컴퓨팅, 심지어 개별 기기 내의 AI 칩까지 포함하는 다층적 컴퓨팅 아키텍처가 발전하고 있습니다.

2.2 빅데이터 수집 기술



빅데이터 분석의 첫 번째 단계는 다양한 소스로부터 데이터를 수집하는 것입니다. 현대의 데이터 생성 환경은 매우 복잡하고 다양하여, 각각의 특성에 맞는 전문화된 수집 기술이 필요합니다. 구조화된 데이터베이스로부터의 정형 데이터 수집부터 실시간 스트리밍 데이터 수집까지, 다양한 수집 방법론과 기술을 체계적으로 살펴보겠습니다.

웹 크롤링과 스크래핑 기술

- 웹 크롤링과 스크래핑은 인터넷상의 방대한 정보를 수집하는 가장 일반적인 방법 중 하나입니다. 이 두 기술은 종종 혼용되지만, 엄밀히 말하면 서로 다른 개념입니다. 웹 크롤링(Web Crawling)은 웹사이트의 구조를 체계적으로 탐색하여 링크를 따라가며 페이지들을 발견하는 과정이고, 웹 스크래핑(Web Scraping)은 특정 웹 페이지에서 필요한 데이터를 추출하는 과정입니다.

웹 크롤링의 기본 원리와 구현

- 웹 크롤링은 시드 URL(Seed URL)에서 시작하여 해당 페이지의 모든 링크를 추출하고, 이를 대기열(Queue)에 추가한 후 순차적으로 방문하는 방식으로 작동합니다. 이 과정에서 중복 URL 제거, 로봇 배제 표준(robots.txt) 준수, 요청 간격 조절 등 다양한 고려사항이 있습니다.
- Python의 Scrapy 프레임워크는 대규모 웹 크롤링을 위한 강력한 도구입니다. 비동기 처리를 기본으로 하여 수천 개의 요청을 동시에 처리할 수 있으며, 미들웨어 시스템을 통해 요청과 응답을 세밀하게 제어할 수 있습니다. Beautiful Soup와 Selenium 같은 도구들은 각각 정적 페이지와 동적 페이지의 데이터 추출에 특화되어 있습니다.
- 대규모 크롤링 작업에서는 분산 아키텍처가 필수적입니다. 여러 대의 크롤링 서버가 URL 대기열을 공유하여 작업을 분담하고, 중앙 관리 시스템이 크롤링 상태를 모니터링하고 조율합니다. Apache Nutch나 StormCrawler 같은 분산 크롤러는 하둡 클러스터 위에서 동작하여 웹 전체를 크롤링할 수 있는 확장성을 제공합니다.

고급 스크래핑 기법들

- 많은 웹사이트들은 JavaScript를 통해 동적으로 콘텐츠를 생성하기 때문에, 단순한 HTML 파싱으로는 원하는 데이터를 추출할 수 없는 경우가 많습니다. 이를 해결하기 위해 **헤드리스 브라우저**(Headless Browser) 기술이 사용됩니다.
- Selenium은 실제 브라우저를 자동화하여 JavaScript 실행 결과까지 포함한 완전한 페이지 렌더링을 수행할 수 있습니다. Puppeteer나 Playwright 같은 최신 도구들은 Chromium 기반의 헤드리스 브라우저를 제어하여 더 빠르고 안정적인 동적 스크래핑을 가능하게 합니다.
- 대규모 스크래핑 작업에서는 IP 차단을 피하기 위한 다양한 기법들이 사용됩니다. 프록시 로테이션(Proxy Rotation)을 통해 요청 출처를 분산시키고, User-Agent 문자열을 무작위로 변경하여 실제 브라우저처럼 보이게 합니다. 또한 요청 간격을 랜덤하게 조절하고, 세션 쿠키를 관리하여 자연스러운 브라우징 패턴을 모방합니다.

윤리적, 법적 고려사항

- 웹 스크래핑은 강력한 데이터 수집 도구이지만, 윤리적, 법적 제약이 따릅니다. 대부분의 웹사이트는 robots.txt 파일을 통해 크롤링 정책을 명시하며, 이를 준수하는 것이 기본

적인 예의입니다. 또한 과도한 요청으로 서버에 부하를 주는 것을 피하고, 저작권이 있는 콘텐츠의 경우 적절한 이용 허가를 받아야 합니다.

- 유럽의 GDPR이나 미국의 CCPA 같은 개인정보보호 규정들은 개인 데이터의 수집과 처리에 엄격한 제한을 두고 있습니다. 따라서 스크래핑 작업을 계획할 때는 수집하는 데이터의 성격을 면밀히 검토하고, 필요한 경우 법적 자문을 구하는 것이 중요합니다.

API를 통한 데이터 연동



API(Application Programming Interface)를 통한 데이터 수집은 웹 스크래핑보다 안정적이고 구조화된 방법입니다. 많은 플랫폼과 서비스들이 개발자를 위한 API를 제공하여 정형화된 방식으로 데이터에 접근할 수 있게 해줍니다.

REST API와 GraphQL

- REST(Representational State Transfer) API는 현재 가장 널리 사용되는 웹 API 아키텍처입니다. HTTP 프로토콜을 기반으로 하여 GET, POST, PUT, DELETE 등의 메서드를 통해 리소스를 조작합니다. JSON 형식의 응답을 제공하는 것이 일반적입니다.
- 대부분의 소셜 미디어 플랫폼들이 REST API를 제공합니다. 트위터의 API를 통해서는 실시간 트윗 스트림, 사용자 프로필, 트렌드 정보 등을 수집할 수 있습니다. 페이스북(메타)의 Graph API는 사용자가 승인한 범위 내에서 소셜 그래프 데이터에 접근할 수 있게 해줍니다.
- GraphQL은 REST의 한계를 보완하는 새로운 API 쿼리 언어입니다. 클라이언트가 필요한 데이터만 정확히 요청할 수 있어 over-fetching이나 under-fetching 문제를 해결합니다.
  GitHub, Shopify, Netflix 등이 GraphQL API를 제공하고 있으며, 복잡한 데이터 관계를 다룰 때 특히 유용합니다.

API 인증과 속도 제한

- 대부분의 API는 남용을 방지하기 위해 인증 시스템과 속도 제한(Rate Limiting)을 구현합니다.
  OAuth 2.0은 가장 일반적인 인증 프로토콜로, 사용자의 명시적 동의 하에 제3자 애플리케이션이 사용자 데이터에 접근할 수 있게 해줍니다.
  API 키 방식은 더 간단한 인증 방법으로, 개발자에게 고유한 키를 발급하여 요청을 식별합니다.
- 속도 제한은 시간당 또는 분당 요청 수를 제한하여 서버 과부하를 방지합니다. 예를 들어, 트위터 API는 15분 동안 300개의 요청만 허용합니다.

이러한 제한을 효율적으로 관리하기 위해 요청 큐잉, 백오프(Backoff) 전략, 여러 API 키 로테이션 등의 기법이 사용됩니다.

실시간 API와 웹 훅

- 전통적인 REST API는 폴링(Polling) 방식으로 작동하여 주기적으로 새로운 데이터를 확인해야 합니다. 하지만 실시간 데이터 수집이 중요한 경우에는 더 효율적인 방법들이 필요합니다.
- 웹훅(Webhook)은 이벤트 기반 데이터 푸시 메커니즘입니다. 특정 이벤트가 발생하면 서비스 제공자가 사전에 등록된 URL로 POST 요청을 보내는 방식입니다.
  GitHub의 웹훅은 코드 푸시, 이슈 생성, 풀 리퀘스트 등의 이벤트를 실시간으로 알려주어 CI/CD 파이프라인을 자동화하는 데 활용됩니다.
- WebSocket API는 클라이언트와 서버 간의 지속적인 양방향 연결을 제공합니다. 암호화폐 거래소의 실시간 가격 데이터, 주식 시장 데이터, 실시간 채팅 메시지 등을 수집할 때 주로 사용됩니다. Coinbase Pro나 Binance 같은 거래소들이 WebSocket API를 통해 밀리세컨드 단위의 실시간 시장 데이터를 제공합니다.
- Server-Sent Events(SSE)는 서버에서 클라이언트로의 일방향 실시간 스트리밍을 제공하는 더 간단한 대안입니다. HTTP를 기반으로 하여 구현이 쉽고, 자동 재연결 기능을 제공합니다.

센서 데이터 수집과 IoT



사물인터넷(IoT)의 급속한 발전으로 센서 데이터 수집이 빅데이터의 주요 원천 중 하나로 부상했습니다. 수십억 개의 센서들이 온도, 습도, 압력, 진동, 위치, 이미지 등 다양한 물리적 현상을 디지털 데이터로 변환하여 전송합니다.

IoT 데이터 수집 아키텍처

- IoT 환경에서의 데이터 수집은 일반적으로 다층 아키텍처를 따릅니다. 최하위 레벨에는 센서와 액추에이터들이 있으며, 이들은 직접 인터넷에 연결되거나 게이트웨이를 통해 연결됩니다. 중간 레벨에는 엣지 컴퓨팅 노드들이 있어 1차 데이터 처리와 필터링을 수행합니다. 최상위 레벨에는 클라우드 기반의 데이터 처리 및 저장 시스템이 있습니다.
- MQTT(Message Queuing Telemetry Transport)는 IoT 환경에서 가장 널리 사용되는 통신 프로토콜입니다. 가볍고 저전력이며, 불안정한 네트워크 환경에서도 안정적으로 작동하도록 설계되었습니다. 발행-구독(Publish-Subscribe) 모델을 사용하여 센서들이 토픽에 데이터를 발행하면, 관심있는 시스템들이 구독하여 데이터를 수신합니다.

엣지 컴퓨팅과 데이터 전처리

- 모든 센서 데이터를 클라우드로 전송하는 것은 네트워크 대역폭과 비용 측면에서 비효율적입니다. 엣지 컴퓨팅은 데이터 생성 지점 근처에서 1차 처리를 수행하여 이러한 문제를 해결합니다.
- 엣지 노드에서는 데이터 필터링, 집계, 압축 등의 전처리 작업이 수행됩니다. 예를 들어, 공장의 진동 센서는 초당 수천 개의 데이터 포인트를 생성할 수 있지만, 엣지에서 이를 통계적으로 요약하여 평균, 최댓값, 표준편차 등의 요약 통계만 클라우드로 전송할 수 있습니다. 이상 상황이 감지된 경우에만 원본 데이터를 전송하는 예외 기반 전송 방식도 널리 사용됩니다.
- Intel의 OpenVINO, NVIDIA의 Jetson, Google의 Edge TPU 같은 엣지 AI 플랫폼들은 기계학습 모델을 엣지에서 실행할 수 있게 해줍니다. 이를 통해 이상 감지, 패턴 인식, 실시간 의사결정 등이 낮은 지연시간으로 가능해집니다.

산업별 센서 데이터 활용 사례

- 제조업에서는 Industry 4.0의 일환으로 스마트 팩토리 구축이 활발합니다. 생산 설비에 부착된 진동, 온도, 전류 센서들이 설비의 상태를 실시간으로 모니터링하고, 머신러닝 알고리즘이 이 데이터를 분석하여 고장을 예측합니다. 이를 통해 예방 정비를 실시하여 가동 중단 시간을 최소화하고 생산성을 향상시킵니다.
- 농업 분야에서는 정밀 농업(Precision Agriculture)을 위해 토양 수분, pH, 영양분 센서와 드론 기반 멀티스펙트럼 카메라를 활용합니다. 이 데이터들을 분석하여 작물별, 구역별로 최적화된 관수와 시비 계획을 수립하여 생산량을 늘리고 자원 사용을 최적화합니다.
- 스마트시티에서는 대기질 센서, 소음 측정기, 교통량 센서, 지능형 가로등 등이 도시 전반의 환경을 모니터링합니다. 이 데이터를 통합 분석하여 교통 신호 최적화, 환경 정책 수립, 공공 서비스 개선 등에 활용합니다.

데이터 품질과 보안 고려사항

- IoT 환경에서는 수많은 센서들이 다양한 환경 조건에서 작동하기 때문에 데이터 품질 관리가 특히 중요합니다. 센서 고장, 네트워크 중단, 간섭 등으로 인한 데이터 손실이나 왜곡이 발생할 수 있습니다. 따라서 중복 센서 배치, 데이터 검증 알고리즘, 이상치 탐지 등의 방법으로 데이터 품질을 보장해야 합니다.
- 보안 측면에서도 IoT는 취약점이 많습니다. 대부분의 IoT 기기들은 제한된 컴퓨팅 자원으로 인해 강력한 암호화를 구현하기 어렵습니다. 또한 물리적으로 접근 가능한 환경에 배치되어 있어 공격에 노출될 위험이 높습니다. 따라서 디바이스 인증, 데이터 암호화, 네트워크 세분화, 정기적 펌웨어 업데이트 등의 보안 대책이 필수적입니다.

데이터베이스 연동과 ETL



기업의 기존 데이터베이스에 저장된 정형 데이터를 빅데이터 분석 환경으로 가져오는 것은 여전히 중요한 데이터 수집 방법입니다. 이 과정에서는 ETL(Extract, Transform, Load) 프로세스가 핵심적인 역할을 합니다.

전통적 데이터베이스 연동

- 관계형 데이터베이스(Oracle, MySQL, PostgreSQL, SQL Server 등)에서 데이터를 추출하는 방법은 여러 가지가 있습니다. 전체 테이블을 한 번에 추출하는 Full Load 방식은 구현이 간단하지만, 데이터양이 많을 때는 시간이 오래 걸리고 시스템 부하를 일으킬 수 있습니다.
- 증분 로드(Incremental Load) 방식은 변경된 데이터만 추출하여 효율성을 높입니다. 타임스탬프 기반 증분 로드는 수정 일시 컬럼을 이용하여 마지막 추출 시점 이후 변경된 레코드만 가져옵니다. 트랜잭션 로그 기반 CDC(Change Data Capture)는 데이터베이스의 트랜잭션 로그를 분석하여 실시간으로 변경 사항을 감지하고 반영할 수 있습니다.
- Apache Sqoop는 하둡 생태계에서 관계형 데이터베이스와 하둡 간의 데이터 전송을 위한 전용 도구입니다. JDBC 드라이버를 통해 다양한 데이터베이스에 연결하고, 병렬 처리를 통해 대용량 데이터를 효율적으로 전송할 수 있습니다.

ETL vs ELT 패러다임

- 전통적인 ETL 프로세스에서는 데이터를 추출(Extract)한 후 변환(Transform)을 거쳐 목적지에 로드(Load)합니다.
  이 방식은 변환 과정에서 데이터 품질을 보장하고 비즈니스 룰을 적용할 수 있다는 장점이 있습니다.
- 하지만 빅데이터 환경에서는 ELT(Extract, Load, Transform) 패러다임이 더 적합한 경우가 많습니다.
  원본 데이터를 먼저 데이터 레이크에 로드한 후 필요에 따라 변환하는 방식입니다. 이는 스키마-온-리드(Schema-on-Read) 접근법으로, 다양한 형태의 데이터를 유연하게 처리할 수 있습니다.
- 최신 클라우드 기반 데이터 웨어하우스들(Snowflake, BigQuery, Redshift 등)은 강력한 컴퓨팅 파워를 제공하여 ELT 방식의 대규모 데이터 변환을 효율적으로 처리할 수 있습니다.

현대적 데이터 통합 도구

- Apache NiFi는 데이터 흐름을 시각적으로 설계하고 관리할 수 있는 강력한 데이터 통합 플랫폼입니다.
  웹 기반 UI를 통해 드래그 앤 드롭으로 데이터 파이프라인을 구성할 수 있으며, 실시간 모니터링과 에러 처리 기능을 제공합니다.
- Talend, Informatica, SSIS 같은 상용 ETL 도구들은 기업급 기능과 지원을 제공합니다. 복잡한 비즈니스 룰 적용, 데이터 품질 관리, 메타데이터 관리 등의 고급 기능을 포함합니다.
- 클라우드 네이티브 도구들도 주목받고 있습니다. AWS Glue, Azure Data Factory, Google Cloud Dataflow 등은 서버리스 환경에서 확장 가능한 데이터 처리를 제공합니다.

스트리밍 데이터 수집



실시간 데이터 처리의 중요성이 커지면서 스트리밍 데이터 수집 기술이 빅데이터 분야의 핵심 기술로 부상했습니다. 스트리밍 데이터는 연속적으로 생성되는 데이터로, 실시간 분석과 대응이 가능해야 합니다.

Apache Kafka 생태계

- Apache Kafka는 현재 가장 널리 사용되는 분산 스트리밍 플랫폼입니다. LinkedIn에서 개발되어 오픈소스로 공개된 Kafka는 높은 처리량, 낮은 지연시간, 확장성, 내결함성을 제공합니다.
  - Kafka의 핵심 개념은 발행-구독 메시징 모델입니다. 프로듀서(Producer)가 토픽(Topic)에 메시지를 발행하면, 컨슈머(Consumer)가 해당 토픽을 구독하여 메시지를 수신합니다. 토픽은 파티션으로 나뉘어 여러 브로커에 분산 저장되어 병렬 처리와 확장성을 제공합니다.
  - Kafka Connect는 외부 시스템과 Kafka 간의 데이터 연동을 위한 프레임워크입니다. 200여 개의 사전 구축된 커넥터를 통해 데이터베이스, 클라우드 스토리지, 메시징 시스템 등과 쉽게 연동할 수 있습니다.
  - Kafka Streams는 Kafka 위에서 실시간 스트림 처리를 위한 라이브러리입니다. 별도의 클러스터 없이 애플리케이션 내에서 스트림 처리 로직을 구현할 수 있어 배

포와 운영이 간편합니다.

다른 스트리밍 플랫폼들

- Apache Pulsar는 Yahoo에서 개발한 차세대 메시징 시스템으로, Kafka의 일부 제약 사항을 해결하고자 합니다. 스토리지와 서빙 레이어를 분리하여 더 나은 확장성을 제공하고, 멀티 테넌시와 지리적 복제 기능을 내장하고 있습니다.
    - Amazon Kinesis는 AWS의 관리형 스트리밍 서비스로, Kinesis Data Streams, Kinesis Data Firehose, Kinesis Analytics 등의 구성 요소를 제공합니다. 완전 관리형 서비스로 인프라 관리 부담 없이 스트리밍 애플리케이션을 구축할 수 있습니다.
    - Google Cloud Pub/Sub는 구글의 메시징 서비스로, 글로벌 규모의 확장성과 강력한 전달 보장을 제공합니다. Firebase와 같은 다른 구글 서비스들과의 긴밀한 연동이 장점입니다.

스트림 처리 패턴들

- 스트리밍 데이터 처리에는 여러 가지 패턴이 있습니다. 윈도우(Windowing) 패턴은 무한한 데이터 스트림을 시간이나 개수 기준으로 나누어 처리합니다.
  텀블링 윈도우(Tumbling Window)는 겹치지 않는 고정 크기 윈도우를, 슬라이딩 윈도우(Sliding Window)는 겹치는 윈도우를 생성합니다.
- 이벤트 시간과 처리 시간의 차이를 다루는 것도 중요한 과제입니다. 네트워크 지연이나 시스템 장애로 인해 이벤트가 순서대로 도착하지 않을 수 있으며, 이를 처리하기 위해 워터마크(Watermark)와 늦은 데이터 처리 전략이 사용됩니다.
- 정확히 한 번 처리(Exactly-once Processing) 보장은 스트림 처리의 중요한 요구사항입니다. 중복 처리나 누락 없이 모든 메시지를 정확히 한 번만 처리하기 위해 멱등성(Idempotency)과 트랜잭션 메커니즘이 사용됩니다.

2.3 빅데이터 저장 시스템

💡 빅데이터의 효과적인 활용을 위해서는 대용량 데이터를 안정적이고 효율적으로 저장할 수 있는 시스템이 필요합니다. 전통적인 관계형 데이터베이스만으로는 빅데이터의 규모와 다양성을 수용하기 어려워, 새로운 저장 패러다임과 기술들이 개발되었습니다.

분산 파일 시스템: HDFS와 그 진화

HDFS의 기본 아키텍처와 설계 철학

- Hadoop Distributed File System(HDFS)은 빅데이터 저장의 혁신을 이끈 분산 파일 시스템입니다.
  구글의 Google File System(GFS) 논문에서 영감을 받아 개발된 HDFS는 "하드웨어 장애는 예외가 아닌 일상"이라는 전제 하에 설계되었습니다.
- HDFS의 아키텍처는 마스터-슬레이브 구조를 따릅니다. NameNode가 마스터 역할을 하며 파일시스템의 메타데이터를 관리하고, 여러 DataNode들이 실제 데이터를 저장합니다.
  파일은 기본적으로 128MB 크기의 블록으로 나뉘어 여러 DataNode에 분산 저장되며, 각 블록은 기본적으로 3개의 복제본을 가집니다.
- 이러한 설계는 몇 가지 중요한 장점을 제공합니다.
  - 첫째, 데이터의 안정성입니다. 하나 또는 두 개의 노드에 장애가 발생해도 데이터 손실 없이 서비스를 계속할 수 있습니다.
  - 둘째, 확장성입니다. 저장 공간이 부족하면 새로운 DataNode를 추가하기만 하면 되므로 페타바이트급까지 쉽게 확장할 수 있습니다.
  - 셋째, 비용 효율성입니다. 고가의 전용 하드웨어 대신 일반적인 서버를 사용하여 구축할 수 있습니다.
- 하지만 HDFS는 대용량 파일의 순차 읽기에 최적화되어 있어, 작은 파일이 많거나 랜덤 액세스가 필요한 경우에는 비효율적입니다.
  또한 NameNode가 단일 장애점이 되는 문제도 초기에는 존재했습니다.

HDFS의 발전과 개선사항

- HDFS는 지속적인 개선을 통해 초기의 한계점들을 극복해왔습니다. Hadoop 2.0에서 도입된 NameNode 고가용성(High Availability)은 능동-대기(Active-Standby) 구조를 통해 단일 장애점 문제를 해결했습니다. 두 개의 NameNode가 ZooKeeper를 통

해 동기화되며, 주 NameNode에 장애가 발생하면 자동으로 대기 NameNode가 서비스를 인계받아 관리합니다.

- HDFS Federation은 여러 개의 독립적인 NameNode가 하나의 클러스터를 관리할 수 있게 해줍니다. 이를 통해 메타데이터 확장성 문제를 해결하고, 각 NameNode가 관리하는 네임스페이스를 분리하여 더 나은 격리성을 제공합니다.

HDFS의 대안들과 차세대 기술

- HDFS의 성공에도 불구하고 새로운 요구사항들이 등장하면서 여러 대안 기술들이 개발되었습니다.
- Apache Ozone은 오브젝트 스토리지와 파일시스템의 장점을 결합한 차세대 스토리지 시스템입니다.
    - 수십억 개의 작은 파일을 효율적으로 처리할 수 있으며, S3 호환 API를 제공하여 클라우드 네이티브 애플리케이션과의 통합이 용이합니다.
- JuiceFS, Alluxio 같은 분산 파일시스템들은 메모리 계층을 활용하여 더 빠른 데이터 액세스를 제공합니다.
  특히 머신러닝 워크로드에서 반복적인 데이터 읽기가 많은 경우 상당한 성능 향상을 제공합니다.

NoSQL 데이터베이스의 다양한 모델



NoSQL(Not Only SQL) 데이터베이스는 관계형 데이터베이스의 ACID 제약을 완화하고 확장성을 우선시하는 데이터베이스 시스템들의 총칭입니다. 데이터 모델에 따라 크게 네 가지 유형으로 분류됩니다.

문서형 데이터베이스 (Document Databases)

- MongoDB는 가장 널리 사용되는 문서형 데이터베이스입니다.
    - JSON과 유사한 BSON(Binary JSON) 형식으로 데이터를 저장하며, 스키마 없이 유연한 데이터 구조를 지원합니다.
    - 문서 내에 중첩된 객체와 배열을 저장할 수 있어 복잡한 데이터 관계를 자연스럽게 표현할 수 있습니다.
    - MongoDB의 분산 아키텍처는 샤딩(Sharding)을 통해 수평적 확장을 지원합니다.

    - 샤드 키를 기준으로 데이터를 여러 샤드에 분산하며, MongoDB가 자동으로 데이터 분산과 쿼리 라우팅을 관리합니다. 복제셋(Replica Set)을 통해 고가용성과 데이터 안정성도 보장합니다.
- Amazon DocumentDB, Azure Cosmos DB 등은 클라우드 환경에서 완전 관리형 문서 데이터베이스 서비스를 제공합니다.
    - 이들은 MongoDB와 호환되는 API를 제공하면서 클라우드의 탄력성과 관리 편의성을 함께 제공합니다.

키-값 저장소 (Key-Value Stores)

- 키-값 저장소는 가장 단순한 NoSQL 모델로, 고유한 키에 값을 매핑하여 저장합니다. 단순함으로 인해 매우 높은 성능과 확장성을 제공할 수 있습니다.
- Redis는 인메모리 키-값 저장소로서 마이크로세컨드 단위의 응답시간을 제공합니다. 문자열, 리스트, 셋, 해시, 정렬된 셋 등 다양한 데이터 타입을 지원하며, Lua 스크립팅, 발행-구독 메시징, 트랜잭션 등의 고급 기능도 제공합니다. 캐싱, 세션 저장, 실시간 순위표, 메시지 브로커 등 다양한 용도로 활용됩니다.
- Amazon DynamoDB는 완전 관리형 NoSQL 서비스로, 서버리스 아키텍처와 자동 확장 기능을 제공합니다. 예측 가능한 성능과 내장된 보안 기능을 통해 미션 크리티컬한 애플리케이션에서도 사용됩니다.

컬럼 패밀리 데이터베이스 (Column Family Databases)



컬럼 패밀리 데이터베이스는 구글의 BigTable 모델을 기반으로 설계되어 대량의 구조화된 데이터를 효율적으로 저장하고 처리할 수 있습니다. 행(Row)과 컬럼으로 구성되지만 관계형 데이터베이스와는 달리 각 행이 서로 다른 컬럼 집합을 가질 수 있어 스키마 유연성을 제공합니다.

- Apache Cassandra는 분산 컬럼 패밀리 데이터베이스의 대표주자입니다.
    - 페이스북에서 개발되어 Apache 재단에 기증된 Cassandra는 선형적 확장성과 고가용성을 제공합니다.
    - 마스터-슬레이브 구조가 아닌 피어-투-피어(Peer-to-Peer) 아키텍처를 사용하여 단일 장애점을 제거했습니다.
    - Cassandra의 데이터 모델은 컬럼 패밀리(Column Family) 또는 테이블로 구성됩니다.

        - 각 행은 파티션 키로 식별되며, 같은 파티션 키를 가진 데이터는 동일한 노드에 저장되어 효율적인 쿼리를 가능하게 합니다.
        - 복합 기본 키를 통해 시계열 데이터나 계층적 데이터를 효과적으로 모델링할 수 있습니다.
        - 튜닝 가능한 일관성(Tunable Consistency)은 Cassandra의 독특한 특징 중 하나입니다.
        - ONE, QUORUM, ALL 등의 일관성 레벨을 선택하여 성능과 일관성 사이의 균형을 조절할 수 있습니다.
        - 이를 통해 애플리케이션의 요구사항에 맞게 CAP 정리의 trade-off를 조정할 수 있습니다.
- HBase는 하둡 생태계의 컬럼 패밀리 데이터베이스로, HDFS 위에서 동작하며 BigTable의 오픈소스 구현체입니다. 강한 일관성을 제공하며, 하둡의 다른 구성 요소들과 긴밀하게 통합됩니다. Phoenix SQL 엔진을 통해 표준 SQL로 HBase 데이터에 접근할 수 있어 기존 SQL 사용자들의 진입 장벽을 낮춰줍니다.

그래프 데이터베이스 (Graph Databases)

- 그래프 데이터베이스는 노드(Node)와 엣지(Edge)로 구성된 그래프 구조로 데이터를 저장하고 처리하는 데이터베이스입니다. 복잡한 관계와 연결성을 가진 데이터를 다루는 데 특화되어 있으며, 소셜 네트워크, 추천 시스템, 사기 탐지, 지식 그래프 등의 분야에서 활용됩니다.
- Neo4j는 가장 널리 사용되는 그래프 데이터베이스 중 하나입니다. Cypher라는 직관적인 쿼리 언어를 사용하며, ASCII 아트와 유사한 구문으로 그래프 패턴을 표현할 수 있습니다. 예를 들어, "MATCH (a:Person)-[:FRIEND]→(b:Person) WHERE a.name='Alice' RETURN b"와 같은 쿼리로 Alice의 친구들을 찾을 수 있습니다.
- Amazon Neptune은 AWS의 완전 관리형 그래프 데이터베이스 서비스로, 속성 그래프와 RDF 그래프 모델을 모두 지원합니다. Gremlin, SPARQL, Cypher 등 다양한 쿼리 언어를 지원하여 기존 애플리케이션과의 호환성을 제공합니다.
- ArangoDB는 멀티모델 데이터베이스로 문서, 키-값, 그래프 모델을 하나의 데이터베이스에서 지원합니다. 이를 통해 다양한 데이터 타입을 하나의 시스템에서 일관성 있게 관리할 수 있어 복잡한 애플리케이션 아키텍처를 단순화할 수 있습니다.

클라우드 스토리지와 데이터 레이크

💡 클라우드 컴퓨팅의 성숙과 함께 클라우드 기반 스토리지 서비스들이 빅데이터 저장의 새로운 패러다임을 제시하고 있습니다.
특히 데이터 레이크(Data Lake) 개념은 다양한 형태의 데이터를 원본 형태로 저장하고 필요에 따라 처리하는 새로운 접근법을 제공합니다.

오브젝트 스토리지의 혁신

- Amazon S3(Simple Storage Service)는 오브젝트 스토리지의 사실상 표준으로 자리잡았습니다.
    - 파일 시스템의 계층적 구조와 달리, 오브젝트 스토리지는 평면적인 네임스페이스에 오브젝트를 저장합니다. 각 오브젝트는 고유한 키로 식별되며, 메타데이터와 함께 저장됩니다.
    - S3의 핵심 특징은 무제한 확장성, 높은 내구성(99.999999999%), 다양한 스토리지 클래스를 통한 비용 최적화입니다.
    - Standard, Infrequent Access, Glacier, Deep Archive 등의 스토리지 클래스를 제공하여 데이터의 액세스 패턴과 비용 요구사항에 맞게 선택할 수 있습니다.
    - Google Cloud Storage는 전역적 일관성과 강력한 보안 기능을 제공합니다. 자동 암호화, IAM 통합, VPC Service Controls 등을 통해 엔터프라이즈급 보안을 보장합니다.
    - 또한 Nearline, Coldline, Archive 스토리지 클래스를 통해 장기 보관 데이터의 비용을 최적화합니다.
- Microsoft Azure Blob Storage는 Hot, Cool, Archive 계층을 제공하며, Azure Active Directory와의 긴밀한 통합으로 하이브리드 클라우드 환경에서의 접근 제어를 간소화합니다.

데이터 레이크 아키텍처

- 데이터 레이크는 정형, 반정형, 비정형 데이터를 원본 형태로 저장하는 중앙 집중식 저장소입니다.
- 전통적인 데이터 웨어하우스가 **스키마-온-라이트(Schema-on-Write) 방식**으로 데이터를 미리 구조화하여 저장하는 것과 달리, 데이터 레이크는 **스키마-온-리드(Schema-on-Read) 방식**으로 분석 시점에 스키마를 적용합니다.
- 이러한 접근법의 장점은 유연성과 비용 효율성입니다. 다양한 형태의 데이터를 수용할 수 있고, 미래의 분석 요구 사항을 미리 예측하지 않아도 됩니다. 또한 오브젝트 스토리지의 낮은 비용으로 대용량 데이터를 경제적으로 저장할 수 있습니다.

- 하지만 잘못 관리되면 데이터 늪(Data Swamp)이 될 위험도 있습니다. 메타데이터 관리, 데이터 카탈로그, 거버넌스 체계 구축이 성공적인 데이터 레이크 운영의 핵심입니다.

클라우드 네이티브 데이터 웨어하우스

- Snowflake는 클라우드 네이티브 데이터 웨어하우스의 선구자로, 컴퓨트와 스토리지를 완전히 분리한 아키텍처를 제공합니다. 이를 통해 독립적인 확장이 가능하며, 사용한 만큼만 비용을 지불하는 종량제 모델을 구현했습니다.
- Google BigQuery는 서버리스 데이터 웨어하우스로, 수 테라바이트 데이터에 대한 쿼리를 몇 초 안에 처리할 수 있는 강력한 성능을 제공합니다. 컬럼형 스토리지와 트리 아키텍처를 통해 대규모 집계 쿼리에 최적화되어 있습니다.
- Amazon Redshift는 페타바이트급 데이터 웨어하우스 서비스로, 기존 PostgreSQL과의 호환성을 유지하면서 분산 처리 성능을 제공합니다. Redshift Spectrum을 통해 S3에 저장된 데이터를 직접 쿼리할 수 있어 데이터 레이크와의 통합이 용이합니다.

2.4 데이터 품질 관리



빅데이터의 가치는 데이터의 품질에 크게 좌우됩니다. 아무리 많은 데이터를 보유하고 있어도 품질이 낮다면 잘못된 분석 결과와 의사결정으로 이어질 수 있습니다. 따라서 체계적인 데이터 품질 관리는 빅데이터 프로젝트 성공의 핵심 요소입니다.

데이터 정제와 전처리

데이터 정제(Data Cleansing)는 데이터세트에서 부정확하거나 부완전한 데이터를 식별하고 수정하는 과정입니다. 실제 분석 프로젝트에서는 전체 시간의 60-80%가 데이터 정제에 소요된다고 알려져 있습니다.

결측 값(Missing Value) 처리

결측값은 데이터 품질 문제의 가장 흔한 형태입니다. 단순한 삭제부터 정교한 통계적 추정까지 다양한 처리 방법이 있습니다.

완전 삭제(Listwise Deletion)는 결측값이 포함된 행을 완전히 제거하는 방법입니다. 구현이 간단하지만 데이터 손실이 크고, 결측값이 무작위가 아닌 경우 편향된 결과를 낳을 수 있습니다.

평균, 중앙값, 최빈값을 사용한 단순 대체는 가장 기본적인 결측값 처리 방법입니다. 수치형 데이터에는 평균이나 중앙값을, 범주형 데이터에는 최빈값을 사용하는 것이 일반적입니다.

더 정교한 방법으로는 회귀 대체(Regression Imputation)가 있습니다. 다른 변수들을 이용해 회귀 모델을 구축하여 결측값을 예측합니다. 다중 대체(Multiple Imputation)는 불확실성을 고려하여 여러 개의 완전한 데이터세트를 생성하고, 각각에 대해 분석을 수행한 후 결과를 통합합니다.

최근에는 머신러닝 기법을 활용한 결측값 처리도 주목받고 있습니다. K-최근접 이웃(KNN), 랜덤 포레스트, 딥러닝 등을 이용하여 복잡한 패턴의 결측값도 정확하게 추정할 수 있습니다.

이상값(Outlier) 탐지와 처리

이상값은 다른 데이터들과 현저히 다른 패턴을 보이는 데이터 포인트입니다. 측정 오류, 입력 오류, 또는 실제 극값일 수 있어 신중한 판단이 필요합니다.

통계적 방법으로는 Z-score, IQR(Interquartile Range), 그러브스 검정(Grubbs' Test) 등이 사용됩니다. Z-score는 평균으로부터 표준편차의 몇 배만큼 떨어져 있는지를 측정하며, 일반적으로 ±3을 임계값으로 사용합니다. IQR 방법은 1분위수와 3분위수를 이용해 이상값 범위를 정의합니다.

머신러닝 기반 이상값 탐지는 더 복잡한 패턴의 이상값을 찾을 수 있습니다. Isolation Forest, One-Class SVM, Local Outlier Factor 등의 알고리즘이 널리 사용됩니다. 딥러닝 모델인 Autoencoder는 정상 데이터의 패턴을 학습하여 그 패턴에서 벗어난 데이터를 이상값으로 탐지할 수 있습니다.

중복 데이터 제거

대용량 데이터에서는 중복 데이터가 빈번하게 발생합니다. 완전히 동일한 레코드는 쉽게 찾을 수 있지만, 부분적으로 다른 중복은 더 복잡한 처리가 필요합니다.

정확 매칭(Exact Match)은 모든 필드가 완전히 일치하는 경우를 찾습니다. 하지만 오타, 형식 차이, 대소문자 차이 등으로 인해 실제로는 같은 데이터임에도 다르게 식별되는 경우가 많습니다.

유사 매칭(Fuzzy Match)은 이러한 문제를 해결하기 위해 사용됩니다. Levenshtein 거리, Jaro-Winkler 거리, 코사인 유사도 등의 알고리즘을 사용하여 문자열 간의 유사도를 측정합니다. 음성학적 알고리즘인 Soundex, Metaphone 등은 발음이 유사한 단어들을 식별할 수 있습니다.

레코드 연결(Record Linkage) 기법은 여러 데이터 소스에서 동일한 엔티티를 식별하는 데 사용됩니다. 확률적 매칭 모델이나 머신러닝 기법을 사용하여 다양한 필드의 정보를 종합적

으로 고려합니다.

데이터 형식 표준화

다양한 소스에서 수집된 데이터는 서로 다른 형식을 가지고 있어 표준화가 필요합니다. 날짜 형식('YYYY-MM-DD', 'MM/DD/YYYY', 'DD/MM/YYYY' 등), 주소 형식, 전화번호 형식, 화폐 단위 등이 대표적인 예시입니다.

정규식(Regular Expression)은 패턴 기반 데이터 변환에 강력한 도구입니다. 복잡한 텍스트 패턴을 식별하고 표준 형식으로 변환할 수 있습니다. 하지만 정규식만으로는 처리하기 어려운 복잡한 경우에는 전용 라이브러리나 API를 사용해야 합니다.

주소 표준화를 위해서는 Google Maps Geocoding API, 우편번호 데이터베이스 등을 활용할 수 있습니다. 회사명 표준화에는 상업적 데이터베이스(Dun & Bradstreet, Bloomberg 등)나 오픈 데이터(OpenCorporates 등)를 활용합니다.

데이터 검증과 모니터링

데이터 품질은 한 번 확보하면 끝나는 것이 아니라 지속적으로 모니터링하고 관리해야 합니다. 특히 실시간으로 유입되는 스트리밍 데이터나 지속적으로 업데이트되는 데이터베이스의 경우 자동화된 검증 시스템이 필수적입니다.

데이터 프로파일링

데이터 프로파일링은 데이터의 구조, 내용, 품질을 체계적으로 분석하는 과정입니다. 각 컬럼의 데이터 타입, 분포, 결측률, 고유값 수, 이상값 등을 종합적으로 파악하여 데이터의 특성을 이해합니다.

구조적 프로파일링(Structural Profiling)은 테이블 구조, 컬럼 정의, 키 관계 등 데이터의 메타구조를 분석합니다. 내용 프로파일링(Content Profiling)은 실제 데이터 값의 분포와 패턴을 분석하며, 관계 프로파일링(Relationship Profiling)은 테이블 간의 참조 무결성이나 함수적 종속성을 검사합니다.

Apache Griffin, Deequ, Great Expectations 같은 오픈소스 도구들이 대규모 데이터에 대한 프로파일링 기능을 제공합니다. 이들은 분산 처리 환경에서 효율적으로 작동하며, 시각화와 리포팅 기능도 함께 제공합니다.

데이터 품질 규칙과 제약조건

효과적인 데이터 검증을 위해서는 비즈니스 규칙을 바탕으로 한 품질 규칙을 정의해야 합니다. 이러한 규칙들은 도메인 제약조건, 형식 제약조건, 일관성 제약조건 등으로 분류할 수 있습니다.

도메인 제약조건은 허용되는 값의 범위를 정의합니다. 예를 들어, 나이는 0-120 범위 내의 정수여야 하고, 이메일 주소는 특정 형식을 따라야 합니다. 정규식, 범위 검사, 목록 검사 등

의 방법으로 구현할 수 있습니다.

참조 무결성 제약조건은 테이블 간의 관계를 검증합니다. 외래 키가 참조하는 기본 키가 실제로 존재하는지, 계층 구조에서 순환 참조가 없는지 등을 확인합니다.

비즈니스 규칙 제약조건은 도메인 지식을 바탕으로 한 복합적인 검증 규칙입니다. 예를 들어, "주문 금액은 주문 항목들의 합계와 일치해야 한다" 같은 규칙입니다.

실시간 데이터 품질 모니터링

스트리밍 데이터 환경에서는 실시간으로 데이터 품질을 모니터링해야 합니다. 배치 처리와 달리 즉시 피드백을 제공하고 필요한 경우 알림을 보내야 합니다.

Apache Kafka Streams, Apache Flink 같은 스트림 처리 엔진을 활용하여 실시간 품질 검증 파이프라인을 구축할 수 있습니다. 데이터가 유입되는 즉시 품질 규칙을 적용하고, 위반사항을 감지하면 별도의 토픽으로 라우팅하거나 알림을 발송합니다.

데이터 드리프트(Data Drift) 탐지도 중요한 모니터링 요소입니다. 시간이 지나면서 데이터의 분포나 패턴이 변화하는 현상으로, 머신러닝 모델의 성능 저하를 야기할 수 있습니다. 통계적 검정(KS-test, Chi-square test 등)이나 분포 비교 방법을 사용하여 탐지할 수 있습니다.

데이터 표준화와 메타데이터 관리

대규모 조직에서는 일관된 데이터 표준과 체계적인 메타데이터 관리가 데이터 품질의 기초가 됩니다. 표준화된 데이터 정의, 코드 값, 형식 등이 있어야 서로 다른 시스템 간의 데이터 연동과 통합 분석이 가능합니다.

데이터 표준화 전략

데이터 표준화는 조직 전체에서 사용하는 데이터 요소들에 대한 공통된 정의와 형식을 수립하는 과정입니다. 마스터 데이터 관리(Master Data Management, MDM)의 일환으로 추진되는 경우가 많습니다.

공통 코드 관리는 표준화의 핵심 요소입니다. 성별, 지역, 업종 등 조직에서 공통으로 사용하는 코드 값들을 중앙에서 정의하고 관리합니다. 코드의 생성, 변경, 폐기에 대한 명확한 프로세스와 승인 체계가 있어야 합니다.

데이터 모델 표준화는 엔티티, 속성, 관계에 대한 일관된 명명 규칙과 정의를 수립합니다. 카멜케이스, 스네이크케이스 등의 명명 규칙, 약어 사전, 금지어 목록 등을 포함합니다.

품질 지표와 임계값 설정도 표준화의 중요한 부분입니다. 완전성, 정확성, 일관성, 유효성, 적시성 등의 품질 차원별로 측정 방법과 허용 가능한 품질 수준을 정의합니다.

메타데이터 관리 시스템

메타데이터는 "데이터에 대한 데이터"로서 데이터의 구조, 의미, 계보, 품질 등에 대한 정보를 포함합니다. 효과적인 메타데이터 관리는 데이터 발견, 이해, 활용을 크게 향상시킵니다.

기술적 메타데이터(Technical Metadata)는 데이터의 물리적 특성을 기술합니다. 테이블 스키마, 컬럼 타입, 인덱스, 파티션 정보 등이 여기에 해당합니다. 대부분은 시스템에서 자동으로 수집할 수 있습니다.

비즈니스 메타데이터(Business Metadata)는 데이터의 비즈니스 의미와 맥락을 기술합니다. 데이터 정의, 비즈니스 규칙, 소유자, 민감도 수준 등이 포함되며, 대부분 수동으로 입력해야 합니다.

운영 메타데이터(Operational Metadata)는 데이터의 생성, 처리, 사용에 관한 정보입니다. 데이터 계보(Data Lineage), 실행 로그, 품질 지표, 사용 통계 등이 여기에 속합니다.

Apache Atlas, LinkedIn DataHub, Amundsen 같은 오픈소스 도구들이 엔터프라이즈급 메타데이터 관리 기능을 제공합니다. 이들은 다양한 데이터 소스로부터 메타데이터를 자동 수집하고, 검색, 브라우징, 협업 기능을 제공합니다.

데이터 카탈로그와 데이터 발견

데이터 카탈로그는 조직의 모든 데이터 자산을 중앙에서 관리하고 검색할 수 있게 해주는 시스템입니다. 구글 검색처럼 키워드로 데이터를 찾고, 메타데이터를 통해 데이터의 의미와 활용 방법을 이해할 수 있게 해줍니다.

현대적인 데이터 카탈로그는 머신러닝을 활용하여 메타데이터를 자동으로 생성하고 추천 기능을 제공합니다. 컬럼명과 데이터 패턴을 분석하여 개인정보가 포함된 컬럼을 자동으로 식별하거나, 사용 패턴을 분석하여 관련 데이터세트를 추천할 수 있습니다.

2.5 데이터 거버넌스와 보안

빅데이터 시대에는 데이터의 양적 증가와 함께 데이터 거버넌스와 보안의 중요성도 기하급수적으로 증가하고 있습니다. 개인정보보호 규제의 강화, 데이터 유출 사고의 증가, 데이터 기반 비즈니스의 확산 등으로 인해 체계적인 데이터 거버넌스 체계 구축이 조직의 생존과 직결되는 문제가 되었습니다.

개인정보보호와 규제 준수

현대 사회에서 개인정보는 새로운 형태의 자산이자 동시에 위험 요소입니다. 적절히 활용하면 혁신적인 서비스와 비즈니스 가치를 창출할 수 있지만, 잘못 관리하면 막대한 법적, 재정적, 평판적 손실을 야기할 수 있습니다.

GDPR과 글로벌 개인정보보호 동향

유럽연합의 일반 데이터 보호 규정(General Data Protection Regulation, GDPR)은 2018년 시행 이후 전 세계 개인정보보호의 새로운 기준점이 되었습니다. GDPR은 개인 데이터의 정의를 매우 포괄적으로 해석하여 직접 식별자뿐만 아니라 간접 식별자, 온라인 식별자(IP 주소, 쿠키 ID 등), 위치 데이터, 생체 인식 데이터 등까지 개인 데이터로 간주합니다.

GDPR의 핵심 원칙 중 하나는 '설계에 의한 프라이버시(Privacy by Design)'입니다. 이는 시스템과 프로세스를 설계할 때부터 개인정보보호를 고려해야 한다는 의미입니다. 데이터 최소화, 목적 제한, 저장 기간 제한, 정확성, 무결성과 기밀성, 책임 등의 원칙을 따라야 합니다.

데이터 주체의 권리 강화도 GDPR의 중요한 특징입니다. 정보에 대한 권리(투명성), 접근권, 정정권, 삭제권(잊힐 권리), 처리 제한권, 데이터 이동권, 반대권, 자동화된 의사결정에 대한 거부권 등을 보장합니다. 특히 '잊힐 권리'는 기술적으로 복잡한 구현을 요구하며, 분산 시스템에서 특정 개인의 모든 데이터를 완전히 삭제하는 것은 매우 어려운 과제입니다.

미국에서는 캘리포니아 소비자 개인정보보호법(CCPA)이 2020년부터 시행되어 미국 내에서도 강력한 개인정보보호 규제가 시작되었습니다. CCPA는 개인정보의 판매에 대한 거부권, 개인정보 공개에 대한 알 권리, 삭제권 등을 규정합니다. 버지니아, 콜로라도, 코네티컷 등 다른 주들도 유사한 법안을 제정하고 있어 미국 전체로 확산되는 추세입니다.

중국의 개인정보보호법(PIPL)은 2021년부터 시행되어 중국 시장에서 활동하는 모든 기업에 영향을 미치고 있습니다. 싱가포르의 개인정보보호법(PDPA), 브라질의 일반 개인정보보호법(LGPD) 등 세계 각국이 GDPR 수준의 강력한 개인정보보호 법안을 도입하고 있습니다.

개인정보 식별과 분류

효과적인 개인정보보호를 위해서는 먼저 조직이 보유한 데이터 중에서 개인정보를 정확히 식별하고 분류해야 합니다. 하지만 빅데이터 환경에서는 수백, 수천 개의 데이터베이스와 파일에 개인정보가 산재해 있어 수동으로 식별하기 어렵습니다.

자동화된 개인정보 식별 도구들이 이러한 문제를 해결하는 데 도움을 줍니다. Microsoft Purview, AWS Macie, Google Cloud DLP API 등은 머신러닝을 활용하여 대용량 데이터에서 개인정보를 자동으로 식별합니다. 이들은 정규식 기반 패턴 매칭, 통계적 분석, 자연어 처리 등을 종합적으로 활용합니다.

개인정보의 민감도에 따른 분류도 중요합니다. 일반 개인정보(이름, 주소, 전화번호), 민감 개인정보(건강 정보, 생체 정보, 종교), 특별 보호 개인정보(주민등록번호, 여권번호, 신용카드번호) 등으로 분류하여 각각에 적합한 보호 수준을 적용해야 합니다.

컨텍스트 기반 분류도 고려해야 합니다. 동일한 데이터라도 사용 목적, 접근 권한, 공유 범위에 따라 위험 수준이 달라집니다. 내부 직원용 보고서의 집계 데이터와 외부 파트너와 공유하는 개별 고객 데이터는 같은 민감도를 가져도 서로 다른 보호 조치가 필요합니다.

데이터 익명화와 가명화 기술

개인정보보호와 데이터 활용 간의 균형을 맞추기 위해 익명화와 가명화 기술이 중요한 역할을 합니다. 하지만 단순한 식별자 제거만으로는 충분하지 않으며, 재식별 공격에 대한 방어도 고려해야 합니다.

k-익명성(k-Anonymity)은 데이터셋의 각 레코드가 최소 k-1개의 다른 레코드와 구별되지 않도록 하는 기법입니다. 준식별자(Quasi-identifier)를 일반화하거나 억제하여 달성할 수 있습니다. 예를 들어, 구체적인 나이를 연령대로 변환하거나, 구체적인 주소를 시/도 수준으로 일반화할 수 있습니다.

l-다양성(l-Diversity)은 k-익명성의 한계를 보완하는 기법입니다. k-익명성만으로는 민감한 속성의 동질성 공격(homogeneity attack)이나 배경 지식 공격(background knowledge attack)을 막을 수 없습니다. l-다양성은 각 동일 집합(equivalence class) 내에서 민감한 속성이 최소 l개의 서로 다른 값을 가지도록 요구합니다.

t-근접성(t-Closeness)은 더 나아가 전체 데이터의 민감한 속성 분포와 각 동일 집합 내의 분포 차이가 임계값 t를 넘지 않도록 제한합니다. 이를 통해 왜곡 공격(skewness attack)과 유사성 공격(similarity attack)을 방어할 수 있습니다.

차분 프라이버시(Differential Privacy)는 수학적으로 엄밀한 프라이버시 보장을 제공하는 최신 기법입니다. 개별 레코드의 존재 여부가 쿼리 결과에 미치는 영향을 제한하여 개인의 프라이버시를 보호합니다. Apple, Google, Microsoft 등이 자사 서비스에 차분 프라이버시를 도입하고 있습니다.

동형암호화(Homomorphic Encryption)는 암호화된 상태에서도 연산을 수행할 수 있게 해주는 혁신적인 기술입니다. 데이터를 복호화하지 않고도 통계 분석이나 머신러닝을 수행할 수 있어, 프라이버시를 완벽하게 보호하면서도 데이터 분석이 가능합니다. 하지만 아직 성능 오버헤드가 크고 지원하는 연산이 제한적이어서 실용화에는 시간이 더 필요합니다.

동의 관리와 투명성

GDPR과 같은 현대적 개인정보보호 법규는 개인의 동의를 매우 엄격하게 규정합니다. 동의는 자유롭게 주어져야 하고, 구체적이어야 하며, 충분한 정보에 기반해야 하고, 명확해야 합니다. 또한 언제든지 철회할 수 있어야 합니다.

동의 관리 플랫폼(Consent Management Platform, CMP)은 이러한 요구사항을 충족하기 위한 기술적 솔루션입니다. 사용자가 어떤 목적으로 어떤 데이터를 어떻게 사용하는지에 대해 세분화된 동의를 관리할 수 있게 해줍니다. 동의 상태 변경 시 관련 시스템들이 실시간으로 업데이트되도록 하는 것이 핵심입니다.

데이터 처리에 대한 투명성도 중요한 요구사항입니다. 개인정보 처리 방침을 명확하고 이해하기 쉽게 작성해야 하며, 데이터가 어떻게 수집되고 처리되는지에 대한 정보를 제공해야 합니다. 일부 조직들은 '영양 성분표'와 같은 형식으로 데이터 사용 현황을 시각화하여 제공하고 있습니다.

데이터 보안과 접근 제어

빅데이터 환경에서의 보안은 전통적인 데이터베이스 보안보다 훨씬 복잡합니다. 분산된 환경, 다양한 데이터 형태, 여러 접근 경로, 대용량 처리 요구사항 등이 보안 복잡성을 증가시킵

니다.

다층 보안 아키텍처

효과적인 빅데이터 보안을 위해서는 다층 방어(Defense in Depth) 전략이 필요합니다. 이는 여러 계층에서 서로 다른 보안 통제를 적용하여 단일 취약점이 전체 시스템을 위험에 빠뜨리지 않도록 하는 접근법입니다.

네트워크 보안 계층에서는 방화벽, 침입 탐지 시스템(IDS), 침입 방지 시스템(IPS), 네트워크 분할(Network Segmentation) 등을 적용합니다. 특히 제로 트러스트(Zero Trust) 아키텍처는 "신뢰하지 않고 검증하라"는 원칙에 따라 모든 네트워크 트래픽을 의심하고 검증합니다.

호스트 보안 계층에서는 안티바이러스, 호스트 기반 침입 탐지 시스템(HIDS), 시스템 강화(System Hardening), 패치 관리 등을 수행합니다. 컨테이너 환경에서는 컨테이너 이미지 스캐닝, 런타임 보안 모니터링도 추가됩니다.

애플리케이션 보안 계층에서는 보안 코딩, 취약점 스캐닝, 웹 애플리케이션 방화벽(WAF), API 게이트웨이 보안 등을 적용합니다. 빅데이터 애플리케이션은 종종 복잡한 분산 아키텍처를 가지므로 마이크로서비스 간 통신 보안도 중요합니다.

데이터 보안 계층에서는 현대적인 데이터 카탈로그는 머신러닝을 활용하여 메타데이터를 자동으로 생성하고 추천 기능을 제공합니다. 컬럼명과 데이터 패턴을 분석하여 개인정보가 포함된 컬럼을 자동으로 식별하거나, 사용 패턴을 분석하여 관련 데이터셋을 추천할 수 있습니다.

협업 기능도 현대적 데이터 카탈로그의 중요한 특징입니다. 사용자들이 데이터에 대한 리뷰, 평점, 태그를 남길 수 있고, 질문과 답변을 통해 지식을 공유할 수 있습니다. 이러한 크라우드소싱 방식은 메타데이터의 품질과 완전성을 지속적으로 개선합니다.